

Probabilistic Circuits as Reasoning Machines in Artificial Intelligence (Part I)



Robert Peharz

Institute of Machine Learning and Neural Computation
Graz University of Technology

Cumulative Habilitation Thesis to obtain the Venia Docendi in
Applied Computer Science

August 2026

Acknowledgements

The people who need to be mentioned foremost are my lovely family, who have patiently endured, sometimes enjoyed, and always supported my academic endeavours: Thank you, Elisabeth, Benjamin, Raffael, and Hannah.

This thesis is about probabilistic circuits and tractable probabilistic models. We are a small but growing community, continuously striving to promote and inspire enthusiasm for our approach to AI and probabilistic modeling. It has been a fun ride, and I hope it continues to be for decades (and centuries) to come. It wouldn't be such a great pleasure to work in such an exciting field without the right allies and peers, some of whom are (in alphabetical order): Cory Butz, Cassio de Campos, YooJung Choi, Alvaro Correia, Nicola Di Mauro, Floriana Esposito, Gennaro Gala, Hong Ge, Robert Gens, Zoubin Ghahramani, Kristian Kersting, Steven Lang, Thomas Liebig, Lorenzo Loconte, Alejandro Molina, Martin Mundt, Jhonatan Oliveira, Franz Pernkopf, Pascal Poupart, Erik Quaeghebeur, Carl Rasmussen, Xiaoting Shao, Karl Stelzner, Pranav Subramani, PingLiang Tan, Martin Trapp, Isabel Valera, Guy Van den Broeck, Fabrizio Ventola, Antonio Vergari, Thomas Wedenig, and Yang Yang.

Furthermore, I would like to thank the Austrian Science Fund (FWF) for supporting my work in the context of the Cluster of Excellence "Bilateral AI" (10.55776/COE12).

Abstract

This cumulative habilitation thesis studies probabilistic circuits (PCs) as a powerful and tractable framework for reasoning and learning under uncertainty in artificial intelligence (AI). It first advocates for probability as a core language for AI, emphasizing its connections to logic and information theory; the conceptual simplicity of probabilistic reasoning—based primarily on the sum and product rules; the parallels between probabilistic inference and human cognition; and the role of probability in optimal decision making. However, probability also faces significant computational challenges, as probabilistic inference is NP-hard in almost all probabilistic models. PCs address these challenges through structural constraints that ensure exact computation of a wide range of inference queries in polynomial time, such as marginals, conditionals, most probable explanations, expectations, and more advanced inference tasks. This thesis synthesizes a decade of research across foundations, algorithmic developments, and empirical validation of PCs. Key contributions highlighted in this work are foundational theory of PCs, Bayesian approaches for learning PCs, scalable implementations and integration with deep learning, hybrid models that combine PCs with intractable models, and connections with symbolic machine learning paradigms.

Table of contents

I	Probabilistic Reasoning and AI	1
1	Introduction	3
1.1	What is AI (roughly)?	3
1.2	The Role of Probability	5
1.3	Why Probabilistic Circuits?	8
2	Basic Probability Theory	11
2.1	Measurable Space	11
2.1.1	Events are Sets	12
2.2	Probability Space	14
2.2.1	Why Sigma-Algebras?	15
2.3	Interpretations of Probability	16
2.4	Fundamental Laws of Probability	18
2.4.1	Law of Total Probability (The Sum Rule)	19
2.4.2	Conditional Probability (The Product Rule)	19
2.4.3	Bayes' Rule (The Rule of Inverse Probability)	21
2.4.4	(Conditional) Independence	22
2.5	Random Variables	23
2.5.1	Distribution of a Random Variable	24
2.5.2	Distribution Functions	24
2.5.3	Multivariate Random Variables and Joint Distributions	26
3	Probabilistic Inference	29
3.1	Marginalization (The Sum Rule)	29
3.2	Conditioning (The Product Rule)	31
3.3	The Chain Rule	33
3.4	Bayes Rule (Law of Inverse Probability)	35
3.5	(Conditional) Independence	35

3.6	Expectations	36
3.7	Most Probable Explanation	39
3.8	Decision Theory	39
4	State of The Art	45
5	Probabilistic Circuits	49
5.1	Basic Definitions of Probabilistic Circuits	51
5.1.1	Decomposability and Smoothness	53
5.1.2	PCs as Hierarchical Mixtures	54
5.2	Marginalization	56
5.3	Conditioning	58
5.4	Expectations and Covariances	61
5.5	Determinism and Most Probable Explanation	63
5.6	Further Structural Properties and Tractable Inferences	64
5.7	Overview of Contributions	65
5.7.1	Foundations and Theory of PCs	66
5.7.2	Bayesian Approaches with PCs	67
5.7.3	Deep Learning and PCs	67
5.7.4	Hybrid Models and Inference	69
5.7.5	PCs and Symbolic Models	71
	References	75

Part I

Probabilistic Reasoning and AI

Chapter 1

Introduction

In the last one or two decades we have witnessed a remarkable explosion of the fields of *artificial intelligence* (AI) and *machine learning* (ML). As research fields, AI and ML have gained tremendous momentum, reflected by an about tenfold increase of papers submitted to leading conferences over the last 10 years. In the digital industries ML is widely applied, ranging from more “classic” applications—such as spam mail detection, optical character recognition, machine translation, computer vision, and recommender systems—to more recent creative applications fuelled by advances in generative modelling, as exemplified by diffusion-based models for images and the prominent ChatGPT system. More generally, AI and ML are perceived as integral parts of the so-called “fourth industrial revolution,” promising a massive acceleration in automation of cognitive tasks and a tight integration of the physical and digital worlds.

1.1 What is AI (roughly)?

While being an active and trendy technology, it is hard to fully specify what AI actually is. According to “Artificial Intelligence: A Modern Approach”, the standard textbook by Russell and Norvig [53], we can organize AI into four categories, spanned by two dichotomies: “human-oriented vs. rational AI” and “thinking vs. acting”. Human-oriented AI strives to imitate abilities of human intelligence, while rational AI formalizes intelligence via some well-principled measure of performance. While there is overlap between the human-oriented and rational approaches to AI, one can think of rational AIs which are rather distinct from human intelligence, and, vice versa, rational approaches might be insufficient to explain or imitate human intelligence. Thus, it makes sense to distinct these two styles of AI. This thesis is located in the “rational AI” segment.

The second dichotomy—“thinking vs. acting”—distinguishes more “internal” abilities such as reasoning and cognition from more “external” abilities such as acting in an environment and making good decisions. This distinction also makes sense, even though “acting” surely requires “thinking” to a certain extent.

While defining AI in a precise way poses some challenges, we can attempt to specify some necessary features or desiderata which any AI system needs to display to a certain degree:

- **Knowledge Representation and Modelling.** It is a save to assume that any AI needs to have the ability to store information. More generally, we might assume a certain level of structural organization of information—broadly denoted as *knowledge representation*—facilitating subsequent tasks of the AI system. Perhaps synonymously to knowledge representation we might also say that an intelligent system needs the ability to represent a *model* of the world in which it is embedded. In a generous interpretation, knowledge representation and models can take various forms, ranging from explicit and formal representations, such as axiomatic systems, to implicit ones such as information stored in neural network weights.
- **Reasoning.** Additionally, an AI system will require the ability to *reason*, i.e., to do something with the information stored and process it into “new” insights. The reasoning machinery will in general depend on the type of knowledge representation. A classical example is propositional logic, which would represent knowledge as an axiomatic system together with known facts and use deductive reasoning to derive “new” facts. A classical illustration is the propositional knowledge base

Socrates is human

All humans are mortal

And the use of the *modus ponens* to conclude that

Socrates is mortal

But also common (systems of) equations such as $E = mc^2$ might be interpreted as a representation of knowledge—relating the three quantities E , m and c —and algebraic manipulation as a reasoning process allowing us, for example, to conclude that $\frac{E}{m} = c^2$.

These two requirements, knowledge representation and reasoning, are arguably necessary features for any AI system. They are probably not sufficient, as they are also fulfilled on a small scale by rather simple systems such as pocket calculators, which are not commonly

perceived as “intelligent.” However, some logic-based AI systems in the early area of AI would also be confined to these two aspects, albeit with a high degree of complexity.

Besides knowledge representation and reasoning one would also include the desiderata of *machine learning*, the nowadays most prominent branch of AI:

- **Learning.** According to Simon [58] learning denotes adaptive changes in a system, enabling to perform tasks more effectively and efficiently. Learning thus includes a certain dynamic element in AI, allowing to automatically modify, extend, or complement the knowledge representation mentioned above. This does not mean that learning happens necessarily online (over the lifetime of the AI)—in contrary, many machine learning systems nowadays learn in an offline fashion. Rather, learning represents a method to incorporate and exploit information which was *not present at design time of the AI*, regardless of whether learning happens online or offline.

The desiderata *knowledge representation*, *reasoning*, and *learning* may serve as a minimalist definition of rational AI or, at the very least, describe its necessary features. Russell and Norvig additionally mention the desiderata *perception* (*computer vision*, *computer audition*), *robotics*, and *natural language processing*. In my own opinion, these are natural requirements but are more features describing human-oriented AI.

This habilitation thesis describes the essence of my work on the “minimalist core notion of rational AI” described above, putting knowledge representation, learning and reasoning in the center of attention.

1.2 The Role of Probability

A central working hypothesis in this work is to recognize *probability* as a rigorous and consistent tool for *reasoning* and *learning*. In the following, I briefly summarize arguments for why probability is an excellent core language for AI.

Simplicity

Similar as the reasoning tools sketched above, propositional logic and (systems of) equations, probabilistic reasoning is based on some few *simple* mathematical principles. Arguably, simplicity is key, when it comes to constructing a long-lasting theory of (artificial) intelligence.

Probabilistic reasoning can essentially be reduced to two simple rules, the *sum rule* and the *product rule* [21]. Semantically, the sum rule, also called *marginalization rule*, allows us to *consistently incorporate* unknown facts in our reasoning process, while incorporating the uncertainty associated with these unknown facts. The semantics of the product rule, also

called *conditioning rule*, is to *inject information* in our reasoning process, that is, basing our reasoning on observed facts. Any probabilistic reasoning task, denoted as *probabilistic inference*, can then be reduced to a well-prescribed cascade of these two rule, allowing us to propagate information from observations to predictions while consistently treating the uncertainties described by our model.

Note that simplicity means here *conceptual* simplicity, not *computational* simplicity. Indeed, probabilistic inference (applying the sum and product rule) is in general a “computational nightmare” as it requires summations or integrations which in general scale exponentially in the problem dimensionality. Working around this computational nightmare is a core goal of *probabilistic circuits*, and hence, of this thesis.

Decision Making

Probabilistic reasoning is tightly connected with *decision making* under uncertainty. A classical line of argument are *Dutch book arguments*, which assert that the betting strategy of an agent (human or artificial) must correspond to *some* probability distribution over gambling outcomes. Otherwise, a combination of bets, a so-called Dutch book, can be constructed which the agent will accept according to its strategy, which, however, will incur guaranteed total loss. Dutch book arguments do not discuss optimality of the agent’s betting strategy, but rather are a sanity check, arguing which kind of strategies are in principle admissible, namely, those based on probabilistic reasoning.

More generally, probability lies at the heart of *decision theory* [4] which defines the notion of *optimal decision* in mere probabilistic terms. Specifically, an optimal decision regarding some target quantity Y is defined as the minimizer of the *expected loss*, that is, the expectation of a some loss function $\ell(y', y)$, expressing the regret when predicting y' while the true value of Y is y . In that sense, knowledge of the true underlying joint distribution together with rigorous probabilistic inference inevitably leads to optimal decision, in the sense of minimal expected (or long-term) regret under this distribution.

Qualitative Similarities to Common Sense Reasoning

Furthermore, probabilistic reasoning shares many qualitative features with (human) common sense reasoning, as discussed by Pearl in his classical book “Probabilistic reasoning in intelligent systems: networks of plausible inference” [39].

First, probabilistic reasoning features *non-monotonic reasoning*, i.e. the effect that the degree of plausibility is not a monotonic function of the amount of available information. For example, common sense judges the statement “Tim can fly” as of low plausibility, but

learning the information that “Tim is a bird” would lead to an increase of this plausibility. If, however, one learns the additional piece of information that “Tim is sick” one would consequently lower this plausibility again. This mechanism of *non-monotonic belief update*, i.e. that the plausibility of a statement is not monotonously increasing or decreasing with the amount of information available, is naturally reflected in probabilistic reasoning. Specifically, *conditioning* (the product rule) displays this feature, as conditional distributions might be arbitrarily different depending on the conditioned information. Several classical AI systems for reasoning under uncertainty were lacking the ability of non-monotonous reasoning [39].

Another canonical example where probabilistic reasoning parallels human cognitive reasoning is the *explaining away phenomenon*: two a-priori unrelated causes become correlated upon observing a common effect. For example, a PhD student in ML might observe that (A) their experiments deliver dissatisfying results. Two reasons for this might be (B) the whole theory and foundations of the work are flawed, or (C) there is a bug in the implementation. A priori, statements B and C can be assumed to be uncorrelated. However, when A is observed, then common sense reasoning suggests that learning C makes B less plausible (bringing some relief to the PhD student), showcasing that B and C have become correlated upon observing A. The explaining away effect, as well as other common sense patterns determining *relevance* among quantities, is well reflected in probabilistic reasoning [39].

Bayesianism and Consistency with Logic

Unlike the classical “hard” reasoning tools, such as logic and systems of equations, probability adds a notion of *uncertainty* which is a pressing requirement imposed by the real world. A nice feature of probability is that, when restricted to the extreme probabilities 0 and 1, probability emulates classical propositional logic. Moreover, if one accepts the assumptions of *Cox’s theorem*, then probability is the *only* sound extension of classical logic to *continuous truth* or *plausibility* values, so that one is de facto forced to accept probability in order to be compatible with logic [25].

Cox’s theorem is often cited as a theoretical underpinning of the *Bayesian approach*, which naturally unifies reasoning and learning. The Bayesian perspective allows us to put probabilities on essentially any uncertain quantities of interest, not only those which can arguably be repeated, as required by the classical frequentist notion of probability.¹ Probabilistic modelling is then but the task of constructing a joint distribution expressing the dependencies and uncertainties between all involved model parameters² and the data. The

¹In a nutshell, the classical dispute between Bayesians and frequentists is about how the notion of probability shall be “correctly interpreted.”

²Here, parameter can mean essentially any unknown quantity of interest, not merely model weights.

typical modelling approach is to specify a *prior distribution* over parameters and a *likelihood* relating parameters with the data. The division into prior and likelihood, however, is usually a rather arbitrary choice [20]. Then, in the Bayesian approach, learning is but probabilistic reasoning (posterior inference) about parameters given data.

Information Theory

Information theory was originally designed as a tool of communication engineering [55], but quickly found applications in many other disciplines, such as biology, psychology, and fundamental physics [24]. The basic insight is to quantify the information content of an event by the *negative logarithm of its probability*. As information content and probabilities are merely non-linear but monotonous transformations (log vs. exp) of each other, they are equivalent descriptions of the same quantity. This equivalence is satisfying, as it indicates that (logs of) probabilities are merely measuring information, or actually lack thereof, serving as a natural description of *uncertainty*.

Here, the reason *why* the information is missing is a subordinate matter and one can avoid vague talk about “true” randomness. Otherwise, one would need to worry whether an opponent’s poker cards are truly random (they are not for them, and I could simply grab the cards and check myself), or the outcome of a game of roulette (which, with sufficiently accurate physical measurement could be predicted arbitrarily well), or the pseudo-random behaviour of a chaotic system which seems random but is actually perfectly deterministic, or the effects on the level of particle physics (Heisenberg’s uncertainty principle introduces a fundamental limit for physical beings). While in these examples that nature of “randomness” seems quite different, this does not matter for modelling and reasoning purposes—one simply quantifies uncertainty, or lack of information, with (log) probabilities. As the information state is generally agent-dependent, this line of argument is reinforcing a Bayesian approach to probabilistic reasoning [34].

This thesis puts forward the working hypothesis that probability is an excellent core language for AI, due to its conceptual simplicity (and hence the possibility to be automated), its ability to express and process uncertainty, as well as many compelling connections to logic, human reasoning, decision theory and information theory.

1.3 Why Probabilistic Circuits?

While probability is arguably an excellent learning and reasoning language for AI, it poses substantial computational challenges. For example, *probabilistic graphical models* (PGM)

[28], a prominent framework for probabilistic AI, are decorated with many hardness results concerning inference and learning. Often, it is relatively easy to construct a PGM accurately reflecting the domain of interest, but in which performing sound probabilistic reasoning (via the sum and product rule) has exponential computational cost in the so-called *tree-width* of the PGM. Roughly speaking, PGMs and other types of probabilistic models might be easy to represent but “hard to run”.

This is in stark contrast to, say, neural networks which are constructed in a way that inference remains easy. Of course, the shortcut here is that neural networks do not perform fully fledged probabilistic reasoning, but only learn a single (or few) reasoning task(s). For example, if one learns a neural classifier predicting class label Y from a feature vector $\mathbf{X}$ by minimizing cross-entropy, one really just estimates the conditional distribution $p_{Y|\mathbf{X}}$ with maximum likelihood, i.e. minimizing the Kullback-Leibler divergence between the model distribution and the empirical data distribution. The very same network, however, would have trouble to predict a portion of the features $\mathbf{X}' \subset \mathbf{X}$ given the rest of the features $\mathbf{X}'' = \mathbf{X} \setminus \mathbf{X}'$ while ignoring the label Y altogether. Such a predictor cannot be derived in a meaningful way from $p_{Y|\mathbf{X}}$. Rather, one requires a model over the *whole joint* $p_{Y,\mathbf{X}}$, which would allow to compute the desired conditional distribution $p_{\mathbf{X}'|\mathbf{X}''}$. Of course, there are ample neural networks representations for whole joint distributions, generally known as (*deep*) *generative models* [65]. However, similar as for PGMs, probabilistic inference is generally hard in these models.

Can we get best of both worlds? That is, can we design models that—like PGMs and other probabilistic models—represent *full* joint distributions over the domain of interest, while at the same time allowing to perform flexible inference with the same ease and computational complexity as neural networks? The answer is **yes, with probabilistic circuits (PCs)**! One way to look at PCs is that they are a kind of hybrid model between PGMs and neural networks. Like PGMs—and unlike “standard” neural nets—they are representations of joint distributions. Unlike PGMs they are able to perform **exact** and **efficient** inference, that is, they compute the sum and product rule—as well as many other inference queries—in essentially the same manner and with the same computational cost as neural networks, i.e. by performing one or a few network passes.

The key term here is **structure**: While PCs can be interpreted as neural networks, they need to obey particular constraints guaranteeing that the desired reasoning operations remain tractable. Interestingly, these structural constraints appear at many places in literature, within the probabilistic modelling setting but also in related disciplines such as logic [16] and databases [36]. Within the probabilistic reasoning domain, these structural patterns appear over and over again, be it in exact inference algorithms such as *variable elimination* [15],

the *junction tree algorithm* [28], and *and/or search spaces* [17] or tractable probabilistic models such as *arithmetic circuits* [14], *sum-product networks* [46], *cutset networks* [48], and *probabilistic sentential decision diagrams* [27]. Really, PCs do not represent a novel model class. Rather, they aim to consolidate these existing lines of research under one umbrella and using one syntax [73, 9] and systematically delineate their differences in terms of structural constraints and corresponding tractable reasoning routines [71]. In a nutshell, PCs are a proposed **lingua franca for tractable probabilistic modelling**.

Evidently, PCs don't offer a free lunch nor do they possess a magical secret sauce to make hard probabilistic inference problems all of the sudden easy. At their core, PCs are models too and serve as an approximations of reality. Their structural constraints ensure that the primary goal—probabilistic inference—remains tractable, but this comes at the cost of limiting the choice of neural network structures. These constraints naturally impose limits on the expressive efficiency of PCs, meaning that some distributions, which can be compactly described as an intractable model, may require exponentially larger representations as a PC. Therefore, the key strategy in PCs is to find the best possible approximation of the data distribution under the constraint that inference remains tractable.

This cumulative habilitation thesis describes my contributions to the field of PCs over the last ten years and is organized as follows: In Chapter 2, I introduce basic concepts of probability theory and probabilistic reasoning. In Chapter 3, I illustrate the idea of probability as elegant and widely applicable reasoning tool on the level of distribution functions. Chapter 4 gives a concise overview of state-of-the-art in probabilistic modeling. In Chapter 5, I introduce the framework of PCs and illustrate the contributions I made to this research field. The relevant publications are listed in Part II.

Chapter 2

Basic Probability Theory

The basic concept in probability is the *probability space*. The definition of probability space is somewhat abstract and practitioners like to quickly forget it as soon as they get exposed to more intuitive notions like *random variables* and *probability densities*. However, it pays off to embrace such foundational concepts to a certain extent. Probability as a mathematical discipline is relatively young, only dating back about 300-400 years, and a firm and concise description of probability was not immediate. Today we have arrived at a compact and rigorous formalisation of probability—the *probability space*—which fits on roughly half a page and accurately describes the nature of probability.

The probability space serves as an insurance and contract: probabilistic methods and models can get fairly abstract and complex and one might get confused about whether what one is doing is still sound. The rule of the game is: can we reduce it back to *some* probability space (or at least prove its existence)? If yes, we can call it probability. If not, it should be called something else. In other words, the probability space is the “Linux kernel” of probabilistic AI: we usually don’t need to work with it, but knowing how it works will make us better Linux users.

In this chapter I just introduce the rudimentary ideas of measure-theoretic probability theory, in order to establish its role as central reasoning instrument in AI. A rigorous treatment can be found for example in [5, 51].

2.1 Measurable Space

The probability space is based on a structured set denoted as *measurable space*.

Definition 1 (Measurable Space). Let Ω be any non-empty set and Σ a sigma-algebra over Ω , i.e. Σ is a set of sub-sets $A \subseteq \Omega$. In the following, let $\bar{A} := \Omega \setminus A$ denote the complementary set of A . The sigma-algebra needs to satisfy the following properties:

- $\Omega \in \Sigma$
- $A \in \Sigma$ implies that $\bar{A} \in \Sigma$ (closed under complement)
- for any countable collection of events $\{A_i\}_i$ it holds that $\bigcup_i A_i \in \Sigma$.
(closed under countable union)

The tuple (Ω, Σ) is a measurable space.

Basically, a measurable space is just a non-empty set (Ω) plus some structure represented as a collection of sub-sets (Σ). The set Ω is called the *sample space* and the elements $\omega \in \Omega$ are called *atomic events*. The basic picture in probability is that an element of ω is selected *at random* from Ω , where “at random” really just means that we are lacking the information which ω is selected. The basic idea of probability is to mathematically model and treat this lack of information.

It is important to note that in general we should *not* assume that an ω is selected *repeatedly* from Ω , as it is assumed in many classical statistics texts, but rather that *exactly one* ω is selected, and we do not know which one. The point here is that assuming repeated selection is unnecessarily limiting the theory, and assuming that exactly one ω is selected is in fact the more general picture. Repeated selection of ω is easily subsumed, by “copying the sample space”, i.e. one defines $\Omega_i = \Omega$ for $i = \{1, 2, \dots\}$ and constructs a new sample space as the Cartesian product of the copies $\Omega' := \times_i \Omega_i$. In this duplicated sample space Ω' , the atomic events are now sequences of draws $\omega' = (\omega_1, \omega_2, \dots)$. This construction is also valid for infinite sequences, i.e. when i ranges over the entire $\mathbb{N}$.

Besides the sample space Ω , the probability space contains a sigma-algebra Σ , a system of subsets A of Ω , which needs to obey the three rules in Definition 1. It follows automatically from these rules, that also (i) $\emptyset \in \Sigma$ and (ii) any countable intersection $\bigcap_i A_i$ of $A_i \in \Sigma$ is again in Σ , i.e. the sigma-algebra is also *closed under countable intersection*.

2.1.1 Events are Sets

The elements of Σ are called *events*. This nomenclature hints that while the atomic events ω represent the “finest resolution” of the universe Ω , the sigma-algebra Σ contains events at a more “macroscopic level”. Example 1 illustrates this concept.

Example 1 (Events for a die throw). *When modelling a die, a suitable sample space is $\Omega = \{1, 2, 3, 4, 5, 6\}$ representing all six possible outcomes of the die throw. We might decide that this is too fine-grained for our purposes and that—for whatever reason—we are just interested in two events, namely (i) whether the die shows a prime number and (ii) whether it shows an odd number.*

The die shows a prime number if it turns out that ω is contained in the set $A_p = \{2, 3, 5\}$ and it shows an odd number if ω is in $A_o = \{1, 3, 5\}$. One can convince oneself that—as soon as the sample space Ω is fixed—indeed any thinkable event can be encoded by whether or not $\omega \in A$ where A is some subset of Ω .

We can find the most concise sigma-algebra containing A_p and A_o by starting with $\Sigma_0 = \{A_p, A_o, \Omega\}$ and iterating

$$\Sigma_{i+1} \leftarrow \Sigma_i \cup \{\bar{A} : A \in \Sigma_i\} \cup \{A \cup B : A, B \in \Sigma_i\}$$

that is, we repeatedly include all complements and pairwise unions of events. This process will—for any finite Ω —converge and deliver the most concise sigma-algebra containing the desired events. Here we get

$$\begin{aligned} \Sigma = & \{\emptyset, \{1\}, \{2\}, \{1, 2\}, \{3, 5\}, \{4, 6\}, \\ & \{1, 3, 5\}, \{1, 4, 6\}, \{2, 3, 5\}, \{2, 4, 6\}, \\ & \{1, 2, 3, 5\}, \{1, 2, 4, 6\}, \{3, 4, 5, 6\}, \\ & \{1, 3, 4, 5, 6\}, \{2, 3, 4, 5, 6\}, \Omega\} \end{aligned}$$

We see that this sigma-algebra does not contain all of the singleton atomic events—it contains $\{1\}$ and $\{2\}$, but for example $\{3\}$ and $\{4\}$ are not contained in Σ . That is, Σ represents a more “macroscopic” view of Ω and contains only A_p and A_o plus all the necessary events to get a valid sigma-algebra.

After fixing Ω , any thinkable event can be described as whether ω is contained in some prescribed set $A \subseteq \Omega$ and hence any such subset A deserves the name “event”. Thus, we will sometimes say things like “event A happens,” meaning really that $\omega \in A$. The basic role of Σ is to collect the events we are interested in. Moreover, the structure of the sigma-algebra just ensures that we have a **closed logical system** [25]:

- The sigma-algebra needs to be closed under complement, which means that for any event we consider, we also must consider the opposite, i.e., that the event does *not* happen. This simply amounts to **logical negation**.

- The sigma-algebra needs to be closed under countable union, i.e., for any countable collection of events we need also to consider the event that any of them happens. This amounts to **logical or**.
- It follows automatically due to De Morgan's laws that the sigma-algebra is closed under *countable intersection*, i.e., for any countable collection of events we need also to consider the event that all of them happen. This amounts to **logical and**.
- Further logical connectives follow, such as closedness under symmetric set difference (**logical xor**).

We can now define the notion of probability space.

2.2 Probability Space

While a measurable space is a non-empty set Ω together with a logical structure Σ of sub-sets, a probability space adds the *probability measure*, a function defined on Σ .

Definition 2 (Probability Space). *Let (Ω, Σ) be a measurable space. Let $\mathbb{P}$ be a function $\Sigma \mapsto [0, 1]$ mapping events to real numbers between 0 and 1, with the properties*

- $\mathbb{P}(\Omega) = 1$
- *for any countable collection of disjoint events $\{A_i\}_i$ it holds that $\mathbb{P}(\bigcup_i A_i) = \sum_i \mathbb{P}(A_i)$ (countable additivity)*

Such function $\mathbb{P}$ is called a probability measure defined on (Ω, Σ) . The triplet $(\Omega, \Sigma, \mathbb{P})$ is called a probability space.

Thus, a probability space equips a measurable space with a function $\mathbb{P}$ which assigns to each event a real number—the *probability of the event*—such that the whole Ω gets assigned 1 and that probabilities of disjoint events add up. Note that from this definition it follows that $\mathbb{P}(\emptyset) = 0$ and $\mathbb{P}(\bar{A}) = 1 - \mathbb{P}(A)$.

The probability space according to Definition 2 fully characterizes the nature of probability as understood by current mainstream mathematics. Any probabilistic argument can be reduced to a sample space Ω , a sigma-algebra Σ over Ω and a probability measure $\mathbb{P}$ defined on Σ . Given that probability is a general and powerful tool, this definition is a truly compressed description of the matter.

2.2.1 Why Sigma-Algebras?

Newcomers to probability often wonder why sigma-algebras are included in the basic definition of probability space. Ultimately, we aim to assign probabilities to events, which is the role of the probability measure. We do we need Σ ? Why can't we simply just consider *all* possible events, i.e. *all* possible subsets $A \subseteq \Omega$? In other words, one would want to—without loss of generality—define $\mathbb{P}$ on the *power set* (set of all subsets):¹

$$2^\Omega := \{A : A \subseteq \Omega\}.$$

In that way, one could omit sigma-algebras altogether and make the basic definition of probability space even simpler. This would surely be the way to proceed if it was mathematically meaningful. Sigma-algebras, if one would want them to express a “coarse-grained” structure, could be included at a later stage, but not in the core definition of probability. And indeed, we can safely define $\mathbb{P}$ on all events, as long as Ω is *countable*, i.e., *finite* or of the same cardinality as the *natural numbers*.

However, it turns out that when Ω is uncountable, e.g. the real numbers $\Omega = \mathbb{R}$, the power set becomes “too big” to be used as sigma-algebra. Even something simple such as a uniform distribution over some interval of the real line cannot be constructed in a consistent way. These troubles are not private to probability theory but go back to measure theory, which is concerned with assigning *measures* such as volume, length, mass—and also probability—to sets. The classical question of measure theory was whether the notion of *length* can be generalized from intervals such as $[a, b]$, which naturally has length $b - a$, to arbitrary subsets on the real line. The answer to this is negative, meaning that there exist sets which cannot be assigned a meaningful length, as it would break one or more desiderata about such length measure (countable additivity, translation invariance etc.). Those non-measurable sets are rather abstract and are constructed in a rather indirect way by employing the *axiom of choice*,² An easy to follow illustration of the problem is given in Rosenthal's book [51].

These *non-measurable* sets should be excluded from the discussion and only *measurable sets* should be included. Exactly this is the primary usage of sigma-algebras and the reason why they are included in the basic definition of probability.

There is a standard choice of sigma-algebra when one is working with the real numbers, i.e., $\Omega = \mathbb{R}$, or the D -dimensional Euclidean space, i.e. $\Omega = \mathbb{R}^D$, namely the *Borel sets*. The Borel sets include any subset one could (constructively) think of (and many more), forming the *Borel sigma-algebra*. They are easiest defined via *generated sigma-algebras*.

¹Note that the power set is indeed a sigma-algebra, namely the largest possible one.

²The existence of non-measurable sets is in fact equivalent to the axiom of choice.

Definition 3 (Generated Sigma-Algebra). *Let E be any set of subsets of Ω . The sigma-algebra generated by E is*

$$\Sigma(E) := \bigcap_{E \subseteq \Sigma} \Sigma,$$

that is, the intersection of all sigma-algebras containing E .

Note that in Definition 3 we do not require any special properties from E , but any collection of subsets of Ω is fine. It is quite easy to prove that $\Sigma(E)$ is indeed a sigma-algebra. Furthermore, we can rightfully call it the smallest possible sigma-algebra containing E , since it contains exactly the sets which need to occur in *all* sigma-algebras containing E .

With this tool at hand, the Borel sigma-algebra can be defined as

Definition 4 (Borel Sets, Borel Sigma-Algebra). *The Borel sigma-algebra on $\mathbb{R}$ is defined as*

$$\mathcal{B} := \Sigma(\{[a, b] \mid a, b \in \mathbb{R}, a \leq b\})$$

that is, the sigma-algebra generated by the set of all closed intervals. Alternatively, one can use the open intervals, half-open intervals, etc., which are all equivalent definitions of $\mathcal{B}$.

A likewise definition can be made for $\mathbb{R}^D$, by using the sigma-algebra generated by (open or closed) hypercubes, or alternatively the set of all open (or closed) balls, etc.

In conclusion of this section: sigma-algebras contain those events one wants to consider and make sure that events are closed under set operations (logic connectives). The main reason that they are in the basic definition of probability space is to “fix a bug” and make the math work.

2.3 Interpretations of Probability

The definition of probability space requires about half a page and precisely sets the formalism and rules to work with probability. While it is a crisp and minimalist description of the nature of probability, it lacks intuition about “what probability actually is”. We might think about probability in the following ways.

- **Relative Frequencies.** Relative frequencies are helpful for interpreting probabilities and follow from the *law of large numbers*. Classically, they served as definition of probabilities, before their formalization as in Definition 2.

Given a probability space $(\Omega, \Sigma, \mathbb{P})$, we might construct n independent copies of it. Intuitively, when figuring the probability space as an urn containing various coloured

balls, we would construct n identical urns and let n independent parties perform the draws. The result is then a sequence (or set) of independent and identically distributed draws $\omega = (\omega_1, \omega_2, \dots, \omega_n) \in \Omega^n$. The law of large numbers now guarantees that given any event $A \in \Sigma$

$$\lim_{n \rightarrow \infty} \frac{\sum_{i=1}^n [\omega_i \in A]}{n} \stackrel{a.s.}{=} \mathbb{P}(A),$$

where $[\cdot]$ denotes the Iverson bracket, evaluating to 1 for true arguments and 0 for false arguments. Hence, $\sum_{i=1}^n [\omega_i \in A]$ is the number of times the atomic event has been picked from A across the independent copies of the probability space. Normalizing by n gives us a relative frequency between 0 and 1. The abbreviation *a.s.* means *almost surely*, that is, the equality holds with probability 1 (in the constructed probability space over sequences ω). Hence, drawing the atomic element repeatedly, independently and in identical manner, the relative frequency of how often event A happens approaches $\mathbb{P}(A)$.

- **Relaxation of Logic Truth Values.** A probability measure assigns a real number to each of the considered events. At least for two of these real numbers we have a concrete interpretation, namely for 0 and 1. The whole sample space Ω , but perhaps also other events A , get assigned probability 1, which is naturally interpreted as the logical value *true*. Conversely, the empty set, but perhaps also other events, get the value 0, naturally interpreted as *false*. No matter which ω is picked, it will certainly be contained in Ω and it will certainly not be in $\emptyset$. Moreover, if we restricted the set of return values of $\mathbb{P}$ to 0 and 1, any propositional logic theory can be emulated by constructing a suitable probability space.

In the general case, where probabilities are between 0 and 1, we might naturally interpret probabilities as *plausibility values*. The closer a probability is to 1 the more plausible is its occurrence and the closer it is to 0 the less plausible its occurrence. The rules of probability further dictate that the probability of a union of disjoint events (amounting to a disjunction of logically incompatible statements) should simply add up, which is reasonable. Furthermore, Cox's theorem [25] asserts that—under particular “common sense assumptions”—probability is de facto the only sound extension of propositional logic to real numbers.

- **Information.** Information theory provides a natural interpretation of probability as information content. Shannon [55] defined the *information content* of an event A as

$$-\log \mathbb{P}(A)$$

based on the desiderata that

- an event which occurs certainly should convey zero information
- less probable events (i.e. events which are “closer” to impossibility $\emptyset$) should convey more information
- information of *independent events*³ should add up

The negative logarithm is the only function satisfying these desiderata, where a degree of freedom concerning the base of the logarithm remains. When using the logarithm to base 2 information content is measured in *bits*. A probability of 0.5, for example the outcome of a fair coin flip, translates to 1 *bit*, making intuitively sense. The logarithm to base e expresses information in *nats*. As information content of an event is a bijective function of its probability, probability and information content are basically the same thing.

None of these interpretations should be treated as the “ultimate correct one”. The question how to “correctly” interpret probabilities has been long a point of dispute between “frequentists” and “Bayesians”. For frequentists, the interpretation as relative frequencies is more appealing, while for Bayesians the interpretation as plausibility values of information content is natural.⁴ It is advisable to move beyond discussions about the “true nature” of probability. Different interpretations of probability—and the ability to switch between them—should be understood as a sign of richness and justification of the theory. Regardless of interpretation, we shall embrace probability as what it is at its core: a consistent and rigorous reasoning calculus under uncertainty, based on a few simple principles.

2.4 Fundamental Laws of Probability

Probabilistic modelling entails constructing a probability space that represents the domain of interest together with any uncertainties, addressing the desideratum of *knowledge representation* in AI, as discussed in Section 1.1. The second desideratum is *reasoning*, i.e. using our knowledge to answer queries of interest. An important requirement of any reasoning tool is *conceptual simplicity*, that is, reasoning primitives should be relatively few and easy to

³The notion of independence will be defined in Section 2.4.4.

⁴The Bayesian camp sometimes called probabilities “subjective beliefs”, which is actually an invitation for being criticised for “not being objective” or “believing instead of knowing”. However, the idea of this name bears wisdom: probabilities represent the information status about a particular domain, which, of course, is always depending on the agent (e.g. I know my cards in poker, but not yours, and vice versa). To reflect this idea, I would suggest the term “agent-dependent information status” rather than “subjective beliefs”.

understand. As we will see in Chapter 3, the core of probabilistic reasoning relies on a few core reasoning primitives, in particular the *sum rule* and *product rule*. We will discuss these rules on the level of *distribution functions* (introduced in Section 2.5.2), but they correspond to more fundamental laws on the level of probability measures. We will briefly discuss these laws in the following.

2.4.1 Law of Total Probability (The Sum Rule)

Let a probability space $(\Omega, \Sigma, \mathbb{P})$ be given and let $B \in \Sigma$ be an event. Let $A_1, A_2, \dots$ be a countable collection of *exclusive* (i.e. $A_i \cap A_j = \emptyset$ for $i \neq j$) and *exhaustive* (i.e. $\cup_i A_i = \Omega$) events, that is, they form a partition of Ω . Let now $B \cap A_i$ be the *joint event* that both B and A_i happen. The *law of total probability* states

$$\mathbb{P}(B) = \sum_i \mathbb{P}(B \cap A_i). \quad (2.1)$$

This law follows directly from the countable additivity property of probability measures (Definition 2) since the joint events $B \cap A_i$ form a partition of B , hence the sum of their probabilities must equal $\mathbb{P}(B)$.

As a reasoning primitive, the law of total probability, or *sum rule*, can be interpreted as **ignoring, accounting for unknown facts or forgetting**. In particular, figure that we have elicited all the probabilities of the joint events $B \cap A_1, B \cap A_2, \dots$, that is, a rather detailed information status about whether B and A_1 happen, B and A_2 happen, etc. By summing over these probabilities, we essentially forget (ignore, account for different possibilities) regarding the exclusive events A_i . Example 2 illustrates the concept.

2.4.2 Conditional Probability (The Product Rule)

The second central reasoning primitive in probability is *conditional probability*.

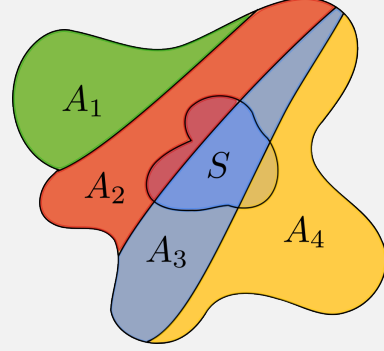
Definition 5 (Conditional Probability). *Given an arbitrary probability space $(\Omega, \Sigma, \mathbb{P})$, let $A, B \in \Sigma$ be two arbitrary events, where $\mathbb{P}(B) > 0$. Then the conditional probability of A given B is*

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}. \quad (2.2)$$

$\mathbb{P}(A|B)$ is simply the probability of A after asserting that $\omega \in B$. In particular, $\mathbb{P}(B|B) = 1$, as should be expected. While the law of total probability (sum rule) shall be interpreted as ignoring, forgetting, or accounting for unknown facts, conditional probability shall be interpreted as **observing or injecting information**. In particular, while the probability space

Example 2 (Law of Total Probability). Consider a sample space Ω representing a population of people, i.e. each $\omega \in \Omega$ represents a person. Let $S \subset \Omega$ be the set of all smokers and further

- $A_1 \subset \Omega$ be the persons of age < 10
- $A_2 \subset \Omega$ be the persons of $10 \leq \text{age} < 20$
- $A_3 \subset \Omega$ be the persons of $20 \leq \text{age} < 50$
- $A_4 \subset \Omega$ be the persons of $50 \leq \text{age}$



Consequently, $\mathbb{P}(S \cap A_1)$ is the probability that somebody smokes and is younger than 10 years, $\mathbb{P}(S \cap A_2)$ the probability that somebody smokes and is between 10 and 20, etc. Rather than keeping all these probabilities, we might decide to forget about age and compute the probability of smoking as

$$\mathbb{P}(S) = \mathbb{P}(S \cap A_1) + \mathbb{P}(S \cap A_2) + \mathbb{P}(S \cap A_3) + \mathbb{P}(S \cap A_4)$$

$(\Omega, \Sigma, \mathbb{P})$ is our “global” mathematical model of uncertainty, conditional probability essentially “rebases” the probability space on B , and updates our model based on the information that $\omega \in B$. This is indeed correct as we can define a *conditional probability space* as follows:

- Define B as the new sample space.
- Define a sigma-algebra Σ' as

$$\Sigma' = \{A \cap B \mid A \in \Sigma\},$$

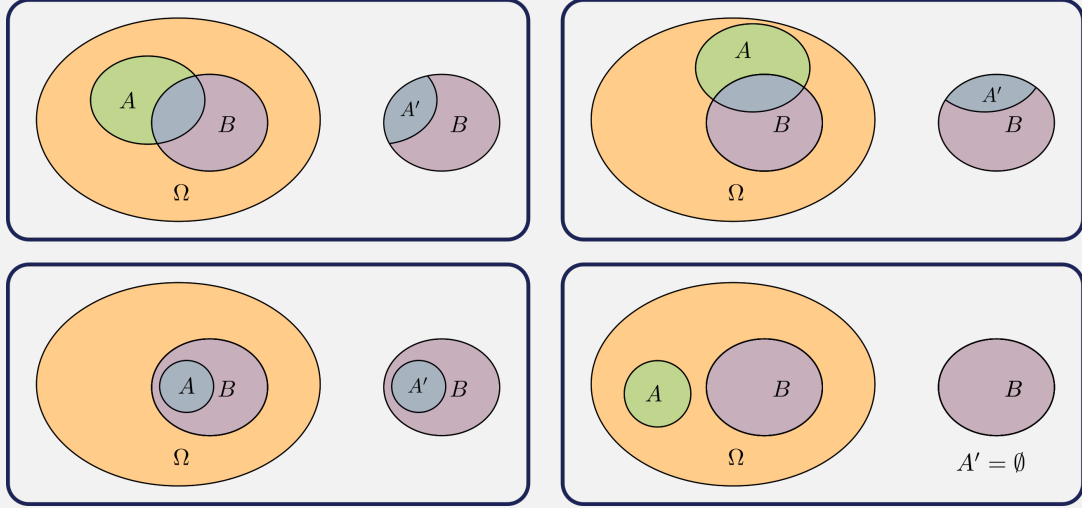
i.e., the collection of the intersection of each original event with B . It can be shown that Σ' is indeed a sigma-algebra over B .

- Define a (conditional) probability measure $\mathbb{P}'$ on Σ' as

$$\mathbb{P}'(A') := \frac{\mathbb{P}(A')}{\mathbb{P}(B)} = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$$

Note that $\mathbb{P}(A')$ is well-defined, since any A' is given as the intersection $A' = A \cap B$ of events in the original Σ . Hence also $A' \in \Sigma$ and $\mathbb{P}(A')$ is defined.

Example 3 (Conditional Probability Space). Let Ω be the sample space of a given probability space and B an event we condition on. The figure below illustrates how various events A get translated into events $A' = A \cap B$ on the conditional probability space defined on B . Conditional probability $\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(A')}{\mathbb{P}(B)}$ can now be interpreted as bona-fide probability measure on the sample space B .



One does not necessarily need to view B as the new sample space, but one can also keep the entire Ω . Of course, the conditional probability for any set A which does not overlap with B must be 0 then.

Then $(B, \Sigma', \mathbb{P}')$ is again a probability space, conditional on $\omega \in B$. Thus, we might see conditional probability as a tool that transforms our original probability space into a new one, accounting the provided information $\omega \in B$. Example 3 illustrates the concept.

2.4.3 Bayes' Rule (The Rule of Inverse Probability)

The product rule immediately delivers *Bayes' rule*. Equation (2.2) can be written as

$$\mathbb{P}(A \cap B) = \mathbb{P}(A|B) \mathbb{P}(B),$$

and by symmetry we get

$$\mathbb{P}(A \cap B) = \mathbb{P}(B|A) \mathbb{P}(A),$$

Equating the right hand sides and dividing by $\mathbb{P}(B)$, we get Bayes' rule:

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(B|A) \mathbb{P}(A)}{\mathbb{P}(B)}. \quad (2.3)$$

Bayes' rule is often called the *law of inverse probability*, as (2.3) essentially reverses the direction of reasoning. Of course, by symmetry we also get

$$\mathbb{P}(B|A) = \frac{\mathbb{P}(A|B) \mathbb{P}(B)}{\mathbb{P}(A)}. \quad (2.4)$$

Bayes' rule is incredibly powerful, since one “conditional direction” (say $\mathbb{P}(A|B)$) is often easy to describe while one is actually interested in the other direction ($\mathbb{P}(B|A)$). This is the general setting of *inverse problems*, which are ubiquitous in science and engineering, such as in imaging, detection, and robotics. Thus, Bayes' rule, while being simple, describes the solution to the vast majority of these problems. However, such *conceptional* simplicity does not mean *computational* simplicity, as most Bayesian problems are numerically challenging.

2.4.4 (Conditional) Independence

Probabilities reflect the information content of an event. Hence, conditional probabilities lead naturally to the notion of *independence*. When we consider an event A whose probability does not change upon observing B , we define that A and B are independent of each other, denoted as $A \perp\!\!\!\perp B$:

$$A \perp\!\!\!\perp B \Leftrightarrow \mathbb{P}(A|B) = \mathbb{P}(A) \quad (2.5)$$

Due to the product rule $\mathbb{P}(A \cap B) = \mathbb{P}(A|B) \mathbb{P}(B)$ we also get

$$A \perp\!\!\!\perp B \Leftrightarrow \mathbb{P}(A \cap B) = \mathbb{P}(A) \mathbb{P}(B) \quad (2.6)$$

and further, since $\mathbb{P}(A \cap B) = \mathbb{P}(B|A) \mathbb{P}(A)$,

$$A \perp\!\!\!\perp B \Leftrightarrow \mathbb{P}(B|A) = \mathbb{P}(B) \quad (2.7)$$

which is the symmetrical case to (2.5). Hence, (2.5), (2.6) and (2.7) are *equivalent definitions* of independence.

Furthermore, independence can be conditional on some other event. In particular, as illustrated Section 2.4.2, conditional probability gives rise to a conditional probability space. By simply applying the concept of independence to the conditional probability space, we get the notion of *conditional independence*. Specifically, let A , B and C be events, where $\mathbb{P}(C) > 0$. Then, A and B are *conditionally independent*, conditionally on C , written as

$A \perp\!\!\!\perp B | C$, if the following equivalent properties hold:

$$A \perp\!\!\!\perp B | C \Leftrightarrow \mathbb{P}(A | B, C) = \mathbb{P}(A | C) \quad (2.8)$$

$$A \perp\!\!\!\perp B | C \Leftrightarrow \mathbb{P}(B | A, C) = \mathbb{P}(B | C) \quad (2.9)$$

$$A \perp\!\!\!\perp B | C \Leftrightarrow \mathbb{P}(A, B | C) = \mathbb{P}(A | C) \mathbb{P}(B | C) \quad (2.10)$$

Note that they are the same as (2.5), (2.6) and (2.7), except that all probabilities are conditioned on C .

2.5 Random Variables

So far, we have discussed probability using its bare-bone measure theoretic definition: We reflect our domain using a (possibly abstract) sample space Ω , which is just a non-empty set. The only thing which “happens” is that an atomic element ω is selected, and we don’t know which one. Our ignorance about which ω is selected is modelled by assigning real numbers—probabilities—between 0 and 1 to all kind of events in a consistent way, namely, that they add up for disjoint events. Probability spaces are the “Linux kernel” of probability, a minimalist and simple theory which, however, unfolds into the rich structure of an intelligent calculus. However, we usually don’t want to work in the Linux kernel, but use an interpretable and high-level interface to work with. The first step towards this interface are *random variables* (RVs).

The basic idea of random variables is to map the abstract sample space Ω into something more interpretable, such as the real numbers $\mathbb{R}$. Thus, a random variable X is really just a function $X: \Omega \mapsto \mathbb{R}$, assigning each $\omega \in \Omega$ to some $X(\omega) \in \mathbb{R}$. Since the selection of ω is random, also the value returned by X will be random, which is why a (deterministic) function defined on a probability space is called a “random variable”.

Evidently, there is no reason to confine random variables to only return real numbers. Rather, depending on the application, one would want them to return complex numbers, tuples of real numbers (vectors), graphs, functions, etc. Thus, a random variable in general is just a transformation of one sample space Ω to some another (interpretable) space $\mathcal{X}$.

In order to consistently work with probability represented by the probability space $(\Omega, \Sigma, \mathbb{P})$, we will require that X is *measurable*, which is defined as follows.

Definition 6 (Measurable Function). *Let (Ω, Σ) and $(\mathcal{X}, \Sigma_X)$ be measurable spaces and let $X: \Omega \mapsto \mathcal{X}$ be a function. X is called measurable if for each $B \in \Sigma_X$ it holds that*

$$X^{-1}(B) \in \Sigma,$$

where X^{-1} is the preimage of B under X , i.e.⁵

$$X^{-1}(B) := \{\omega \in \Omega \mid X(\omega) \in B\}.$$

Measurability of a function requires that both domain Ω and co-domain $\mathcal{X}$ are equipped with sigma-algebras Σ and Σ_X , respectively, and that the pre-images of the events in Σ_X are all contained in Σ . It is thus a condition on the function itself as well as the measurable spaces (Ω, Σ) and $(\mathcal{X}, \Sigma_X)$. Often one would fix the function of interest and design the sigma algebras such that the function indeed becomes measurable.

The defining part of a random variable X is that it is a function defined on a probability space, mapping into a “more interpretable space” $\mathcal{X}$, often called the *state space* of X . Since the selection of $\omega \in \Omega$ is uncertain also the value (state) of $X(\omega) \in \mathcal{X}$ is uncertain. This “transformed” uncertainty is described by the distribution of a random variable.

2.5.1 Distribution of a Random Variable

Given a probability space $(\Omega, \Sigma, \mathbb{P})$ and a random variable X defined on this space and mapping into the measurable space $(\mathcal{X}, \Sigma_X)$, we can define the following function:

$$\mathbb{P}_X: \Sigma_X \mapsto [0, 1], \quad \text{where for any } B \in \Sigma_X: \mathbb{P}_X(B) := \mathbb{P}(X^{-1}(B)).$$

Function $\mathbb{P}_X$ just assigns to each event $B \in \Sigma_X$ the probability which is assigned by $\mathbb{P}$ to the preimage $X^{-1}(B)$, i.e. the set of all ω 's mapping to B . Since the random variable is measurable it is guaranteed that $X^{-1}(B) \in \Sigma$, hence $\mathbb{P}_X$ is properly defined. It can be shown that $\mathbb{P}_X$ is indeed a probability measure, now defined on $(\mathcal{X}, \Sigma_X)$, and hence $(\mathcal{X}, \Sigma_X, \mathbb{P}_X)$ is a probability space. $\mathbb{P}_X$ is called the *distribution induced by X* , the *law of X* , or the *pushforward measure* of $\mathbb{P}$ under X , and it adequately describes our uncertainty about the value $X(\omega)$.

2.5.2 Distribution Functions

The introduction of random variables has just transformed the uncertainty over an “uninterpretable” space Ω into uncertainty over a “more interpretable” space $\mathcal{X}$. The latter is still described with a probability measure which is a rather cumbersome mathematical object. One also might ask why we did the hassle to first define a probability space $(\Omega, \Sigma, \mathbb{P})$ and a measurable function $X: \Omega \mapsto \mathcal{X}$ just to end up with a new probability space $(\mathcal{X}, \Sigma_X, \mathbb{P}_X)$

⁵The symbol for pre-image is the same as for *inverse function*, but these two are distinct concepts. In particular, a function might or might not have an inverse, but the pre-image is always defined.

describing the uncertain quantity X . On this abstract level, it would be more straightforward to construct our model $(\mathcal{X}, \Sigma_X, \mathbb{P}_X)$ directly. In the following, we will thus ignore the underlying probability space $(\Omega, \Sigma, \mathbb{P})$ and just aim to describe $(\mathcal{X}, \Sigma_X, \mathbb{P}_X)$ directly by the means of *distribution functions*. The two most widely used distribution functions are *probability mass functions* and *probability density functions*.

Definition 7 (Probability Mass Function (PMF)). *Let X be a random variable with a countable state space $\mathcal{X}$,⁶ equipped with sigma-algebra $\Sigma_X = 2^{\mathcal{X}}$. The probability mass function of X is defined as*

$$p_X: \mathcal{X} \mapsto \mathbb{R}, \quad p_X(x) = \mathbb{P}_X(\{x\})$$

which assigns to each state $x \in \mathcal{X}$ the probability that $X = x$.

A PMF is a simple and direct way to specify probability distributions. In particular, the probability of any event $A \subseteq \mathcal{X}$ is simply

$$\mathbb{P}_X(A) = \sum_{x \in A} p_X(x), \quad (2.11)$$

since any $A \subseteq \mathcal{X}$ is countable and its probability must be given by the countable sum over the probabilities of the states in A , see Definition 2. However, PMFs can only be defined for discrete random variables, as they assign probabilities in a “discrete” way to each atomic state $x \in \mathcal{X}$. This requires that we assume the power set $2^{\mathcal{X}}$ as sigma-algebra, which is fine for discrete state spaces but does not work for uncountable state spaces, such as $\mathbb{R}$ or $\mathbb{C}$. For such random variables one commonly uses *probability densities* instead.

Definition 8 (Probability Density Function (PDF)). *Let X be a random variable with the real numbers as state space, $\mathcal{X} = \mathbb{R}$, equipped with some sigma-algebra Σ_X (usually the Borel sets) and probability measure $\mathbb{P}_X$. Let p_X be a function mapping from $\mathcal{X}$ to the non-negative real numbers, $p_X: \mathcal{X} \mapsto [0, \infty]$ with the property that*

$$\forall A \in \Sigma_X: \mathbb{P}_X(A) = \int_A p_X(x) dx, \quad (2.12)$$

that is, for any event $A \in \Sigma_X$ the probability $\mathbb{P}_X(A)$ is given by the integral of p_X over A . Then p_X is called a probability density function of X , also just probability density or density. A random variable which admits a density is called continuous.

The probability density, if it exists, is not unique. In particular, any density p_X can be changed at a countable collection of points without changing the integral (2.12) for any A .

⁶A random variable with countable state space is called *discrete*.

Note that both PMFs and PDFs are denoted by the same symbol p_X . The intuitive reason for this is that both serve the same purpose, namely to represent a probability measure $\mathbb{P}_X$ of a random variable X , either via a sum (2.11) or an integral (2.12). Moreover, within a measure-theoretic treatment of probability these two concepts become in fact the same. Specifically, when μ is any measure defined on $(\mathcal{X}, \Sigma_X)$ and $p_X: \mathcal{X} \mapsto [0, \infty]$ is a measurable function with

$$\forall A \in \Sigma_X: \mathbb{P}_X(A) = \int_A p_X d\mu \quad (2.13)$$

then p_X is called a *Radon–Nikodym derivative* of $\mathbb{P}_X$ with respect to μ . In that sense, a PMF is just a Radon–Nikodym derivative with respect to the *counting measure* and a PDF is just a Radon–Nikodym derivative with respect to Lebesgue measure (capturing the notion of length, area, volume, etc.). For brevity, we might just use the shorter term *density* for p_X and also be lose about whether we speak about a discrete random variable (hence using sums) or a continuous random variable (hence using integrals).

2.5.3 Multivariate Random Variables and Joint Distributions

So far we were concerned mainly with univariate random variables, returning a “single” value. However, the usual tasks in science and engineering are to describe how *multiple* quantities are entangled with each other.

We again consider some (abstract) probability space $(\Omega, \Sigma, \mathbb{P})$ and define now *multiple* random variables on it. We might call them X, Y, Z or more systematically enumerate them as $X_1, X_2, \dots, X_D$, where $D \in \mathbb{N}$, and let $\mathcal{X}_1, \mathcal{X}_2, \dots, \mathcal{X}_D$ be their state spaces. Now, since the mental picture is that some atomic event $\omega \in \Omega$ is selected, where $\mathbb{P}$ describes our ignorance about this selection, the returned values $X_1(\omega), X_2(\omega), \dots, X_D(\omega)$ will be uncertain but (in general) also correlated with each other.

The discussion of multivariate random variables really does not add much mathematical depth to univariate random variables. In fact, one simply defines a *random tuple* $\mathbf{X}$ (also often referred to as *random vector* or *set of random variables*) as the function

$$\mathbf{X}: \Omega \mapsto \mathcal{X}, \quad \mathbf{X}(\omega) := (X_1(\omega), X_2(\omega), \dots, X_D(\omega))$$

mapping from Ω to the Cartesian product of the state spaces $\mathcal{X} := \times_{i=1}^D \mathcal{X}_i$ and whose i^{th} component is the univariate random variable X_i . In order to assert that $\mathbf{X}$ is a (vector-valued) random variable one needs to ensure that $\mathbf{X}$ is measurable whenever the univariate random variables $X_1, X_2, \dots, X_D$ are measurable. Indeed, one can show that $\mathbf{X}$ is measurable with respect to the so-called *product sigma-algebra* $\Sigma_{\mathbf{X}}$ which is the sigma-algebra generated by

$E = \left\{ \times_{i=1}^D A_i \mid A_i \in \mathcal{X}_i \right\}$, i.e. the set of all Cartesian products of any events of the univariate random variables. Thus, $\mathbf{X}$ can be understood as a standard random variable, albeit one mapping to the multi-dimensional space $\mathcal{X}$ and equipped with product sigma-algebra $\Sigma_{\mathbf{X}}$.

The concept of distribution functions carries over to multivariate random variables as follows.

Definition 9 (Joint Probability Mass Function (Joint PMF)). *Let $\mathbf{X} = (X_1, \dots, X_D)$ be a random vector, consisting of discrete univariate random variables $X_1, \dots, X_D$ with countable state spaces $\mathcal{X}_1, \dots, \mathcal{X}_D$. The joint probability mass function is defined as*

$$p_{\mathbf{X}}: \mathcal{X} \mapsto [0, 1], \quad p_{\mathbf{X}}(\mathbf{x}) = \mathbb{P}_{\mathbf{X}}(\{\mathbf{x}\})$$

which assigns to each joint state $\mathbf{x}$ the probability that $\mathbf{X} = \mathbf{x}$.

Similarly to univariate random variables the joint PMF describes the probability measure of $\mathbf{X}$ via

$$\mathbb{P}_{\mathbf{X}}(A) = \sum_{\mathbf{x} \in A} p_{\mathbf{X}}(\mathbf{x}). \quad (2.14)$$

for any event $A \in \Sigma_{\mathbf{X}}$. Furthermore, also the concept of PDF carries over to the multivariate case.

Definition 10 (Joint Probability Density Function (Joint PDF)). *Let $\mathbf{X} = (X_1, \dots, X_D)$ be a random vector, consisting of univariate random variables $X_1, \dots, X_D$ each with state space $\mathbb{R}$ and equipped with the Borel sigma-algebra. Let $p_{\mathbf{X}}$ be a function mapping from $\mathbb{R}^D$ to the non-negative real numbers, $p_{\mathbf{X}}: \mathbb{R}^D \mapsto [0, \infty]$, with the property that*

$$\mathbb{P}_{\mathbf{X}}(A) = \int_A p_{\mathbf{X}}(\mathbf{x}) \, d\mathbf{x}, \quad (2.15)$$

for any event (Borel set) $A \subseteq \mathbb{R}^D$. Then $p_{\mathbf{X}}$ is called a joint probability density of $\mathbf{X}$.

The integral (2.15) (or sum (2.14)) is the defining part of a joint distribution function $p_{\mathbf{X}}$: when integrating (or summing) it over an event $A \in \Sigma_{\mathbf{X}}$, it needs to return $\mathbb{P}_{\mathbf{X}}(A)$. From this perspective, one can also define *hybrid* distribution functions over sets of random variables $\mathbf{X}$ which contains both discrete and continuous ones.

Definition 11 (Hybrid Distribution Function). *Let $\mathbf{X} = (X_1, \dots, X_D)$ be a random vector where $\mathbf{X}' = (X_1, \dots, X_K)$ are discrete random variables and $\mathbf{X}'' = (X_{K+1}, \dots, X_D)$ are continuous random variables,⁷ and let $\mathcal{X} = \times_{i=1}^D \mathcal{X}_i$ be their joint state space.*

⁷That the first K random variables are discrete and the others continuous is merely an assumption for convenience of notation, which can be made without loss of generality.

For any event $A \in \Sigma_{\mathbf{X}}$ let

$$A' := \{(x_1, \dots, x_K) \mid (x_1, \dots, x_D) \in A\}$$

i.e., the set of all states assumed by the discrete random variables in A . Furthermore, for each $\mathbf{x}' \in A'$ let

$$A''_{\mathbf{x}'} := \{(x_{K+1}, \dots, x_D) \mid (x_1, \dots, x_D) \in A \wedge (x_1, \dots, x_K) = \mathbf{x}'\}$$

i.e., the set of all states assumed by the continuous random variables assumed in A , whenever the discrete random variables assume state $\mathbf{x}'$.

Let $p_{\mathbf{X}}: \mathcal{X} \mapsto [0, \infty]$ be a function with the property that for any $A \in \Sigma_{\mathbf{X}}$

$$\mathbb{P}_{\mathbf{X}}(A) = \sum_{\mathbf{x}' \in A'} \int_{A''_{\mathbf{x}'}} p_{\mathbf{X}}(\mathbf{x}', \mathbf{x}'') d\mathbf{x}''. \quad (2.16)$$

Then $p_{\mathbf{X}}$ is called a hybrid distribution function over $\mathbf{X}$.

The definition of hybrid distribution functions is a bit cumbersome, as it requires us to split random variables and their state spaces into discrete and continuous ones. Furthermore, all the joint distributions—joint PMFs, joint PDFs, and hybrid distribution functions—can be understood as densities in a measure-theoretic sense, as in (2.13). The only difference between these is just the base measure: for discrete random variables one uses a counting measure, for continuous random variables one typically uses the Lebesgue measure, and for the hybrid case one would use an adequate product measure of counting and Lebesgue measure.

Hence, similarly as in the univariate case we use the same symbol $p_{\mathbf{X}}$ for joint PMFs, joint PDFs and hybrid distribution functions, as they all serve the same purpose—describing a probability measure $\mathbb{P}_{\mathbf{X}}$ by integrating (or summing) $p_{\mathbf{X}}$ over some event A .

Chapter 3

Probabilistic Inference

As discussed in Section 1.1, a core property of any AI system is to have some representation of a *knowledge base*. This role is now fulfilled by distribution functions $p_{\mathbf{X}}$ introduced in the last chapter, representing both *dependencies between random variables* as well as our *uncertainty* about them. Besides representing our knowledge as a distribution function $p_{\mathbf{X}}$ we ultimately are interested in *reasoning*, or *inference*, i.e. answering queries of interest. Probability naturally offers *inference routines* in order to perform reasoning under uncertainty. The two most important routines are **marginalization (the sum rule)** and **conditioning (the product rule)**, where the latter just uses the former as a sub-routine. Hence, marginalization is really the core business of probabilistic inference.

Besides these two central inference routines there are several other fundamental operations one would want to do with probability distributions, in particular taking **expectations** and performing **optimization**, i.e. taking maxima or minima of functions of interest. As it turns out, however, most models mentioned above struggle with computing these queries, as they are typically NP-hard problems in these. **Probabilistic circuits (PCs)**, which we introduce in Chapter 5, will allow to compute these queries **exactly** and **efficiently**.

Before diving into PCs, however, in this chapter we shall study the elegant reasoning calculus probability has to offer, disregarding computational issues. In fact, one of the things which make probabilistic models so interesting is the tension between the **conceptual** simplicity of the calculus—it is in principle clear *what* shall be done—and the **computational hardness** involved when implementing the calculus.

3.1 Marginalization (The Sum Rule)

Let a distribution function $p_{\mathbf{X}}$ for a random vector $\mathbf{X} = (X_1, \dots, X_D)$ be given and let $\mathbf{Y} = (Y_1, \dots, Y_{D'})$ and $\mathbf{Z} = (Z_1, \dots, Z_{D''})$ be a partition of $\mathbf{X}$, i.e. $\mathbf{Y} \cup \mathbf{Z} = \mathbf{X}$ and $\mathbf{Y} \cap \mathbf{Z} = \emptyset$. We

assume without loss of generality that $\mathbf{Y}$ are the first D' entries in $\mathbf{X}$ and $\mathbf{Z}$ the remaining $D'' = D - D'$ entries, i.e.

$$\mathbf{X} = (Y_1, \dots, Y_{D'}, Z_1, \dots, Z_{D''}).$$

The *marginal distribution* $p_{\mathbf{Y}}$ is given as

$$p_{\mathbf{Y}}(y_1, \dots, y_{D'}) = \int \cdots \int p_{\mathbf{X}}(y_1, \dots, y_{D'}, z_1, \dots, z_{D''}) dz_1 \dots dz_{D''}, \quad (3.1)$$

that is, we *integrate out* or *marginalize*¹ all random variables in $\mathbf{Z}$, in order to get a “reduced model” $p_{\mathbf{Y}}$ only over random variables $\mathbf{Y}$. We often write (3.1) more compactly as

$$p_{\mathbf{Y}}(\mathbf{y}) = \int p_{\mathbf{X}}(\mathbf{y}, \mathbf{z}) d\mathbf{z}, \quad (3.2)$$

where we “unpack” the values $\mathbf{y}, \mathbf{z}$ as arguments in $p_{\mathbf{X}}(y_1, \dots, y_{D'}, z_1, \dots, z_{D''})$ and $\mathbf{y}$ as arguments in $p_{\mathbf{Y}}(y_1, \dots, y_{D'})$. While the full joint distribution $p_{\mathbf{X}}$ captures the dependency structure and uncertainties among *all* $\mathbf{X}$, the marginal (3.2) accurately describes dependencies and uncertainties among subset $\mathbf{Y}$, while having eliminated $\mathbf{Z}$ from consideration. Thus, the marginalization operation (3.2) shall be interpreted as the reasoning primitive **ignoring** or **accounting for unknown quantities $\mathbf{Z}$** . Since this happens by integrating or summing over all possible states of the unknown quantity $\mathbf{Z}$, this operation is also often called the **sum rule**. Basically, it is the *law of total probability* (see Section 2.4.1) applied to distribution functions.

A few things are worth mentioning. First, note that $p_{\mathbf{Y}}$ is in general a *joint* distribution, but one over a subset of $\mathbf{X}$. Of course, whenever $\mathbf{Y} = \{Y_i\}$ is singleton, $p_{\mathbf{Y}}$ becomes a univariate distribution p_{Y_i} . The marginal distribution is indeed “the” distribution over $\mathbf{Y}$ and not some kind of approximation. Specifically, just as $p_{\mathbf{X}}$ accurately describes a set of random variables $\mathbf{X}$, $p_{\mathbf{Y}}$ accurately describes $\mathbf{Y}$, “as if $\mathbf{Z}$ had never been considered”.

Also the extreme case of marginalization where all variables $\mathbf{X}$ are marginalized is interesting, as it delivers the *normalization constant* or *partition function*

$$\mathcal{Z} = \int p_{\mathbf{X}}(\mathbf{x}) d\mathbf{x}. \quad (3.3)$$

¹Recall that we do not make much difference whether random variables are discrete or continuous and that for discrete random variables we simply replace an integrals in (3.1) by sums.

The normalization constant is important for *unnormalized models* which represent distribution functions via some nonnegative function $f_{\mathbf{X}}$:

$$p_{\mathbf{X}}(\mathbf{x}) := \frac{f_{\mathbf{X}}(\mathbf{x})}{\mathcal{Z}}, \quad \mathcal{Z} = \int f_{\mathbf{X}}(\mathbf{x}) d\mathbf{x}. \quad (3.4)$$

Furthermore note that marginalization is a *consistent* reasoning pattern, meaning that one always arrives at the same result for any sequence of marginalization operations expressing the same marginal. For example, when starting with a joint $p_{\mathbf{X},\mathbf{Y},\mathbf{Z}}$ we will arrive at the same marginal $p_{\mathbf{X}}$ whether we are (i) first marginalizing $\mathbf{Z}$ (yielding $p_{\mathbf{X},\mathbf{Y}}$) and then $\mathbf{Y}$, or (ii) first marginalizing $\mathbf{Y}$ (yielding $p_{\mathbf{X},\mathbf{Z}}$) and then $\mathbf{Z}$.

3.2 Conditioning (The Product Rule)

Let again a distribution function $p_{\mathbf{X}}$ for a set of random variables $\mathbf{X} = (X_1, \dots, X_D)$ be given and let $\mathbf{Y} = (Y_1, \dots, Y_{D'})$ and $\mathbf{Z} = (Z_1, \dots, Z_{D''})$ be a partition of $\mathbf{X}$. Given a joint state $\mathbf{z}$ for $\mathbf{Z}$, the *conditional distribution* of $\mathbf{Y}$ given $\mathbf{z}$ is

$$p_{\mathbf{Y}|\mathbf{Z}}(\mathbf{y}|\mathbf{z}) = \frac{p_{\mathbf{X}}(\mathbf{y}, \mathbf{z})}{p_{\mathbf{Z}}(\mathbf{z})} = \frac{p_{\mathbf{X}}(\mathbf{y}, \mathbf{z})}{\int p_{\mathbf{X}}(\mathbf{y}, \mathbf{z}) d\mathbf{y}}, \quad (3.5)$$

i.e., the ratio of the joint distribution $p_{\mathbf{X}}$ and the marginal $p_{\mathbf{Z}}$, where the latter is derived from $p_{\mathbf{X}}$ using the sum rule. The conditional distribution is the concept of conditional probability (see Section 2.4.2) applied to distribution functions.

A natural way to interpret $p_{\mathbf{Y}|\mathbf{Z}}$ is as a *family of distribution functions* over $\mathbf{Y}$, indexed by the states $\mathbf{z} \in \mathcal{Z}$. While the full joint distribution $p_{\mathbf{X}}$ captures the dependencies and uncertainties among *all* random variables $\mathbf{X}$, the conditional distribution $p_{\mathbf{Y}|\mathbf{Z}}$ describes dependencies and uncertainties among $\mathbf{Y}$ upon establishing the event $\mathbf{Z} = \mathbf{z}$. Hence, the conditioning operation (3.5) shall be interpreted as the reasoning primitive **observing** or **injecting evidence** (that $\mathbf{Z} = \mathbf{z}$) and updating the distribution over the remaining random variables $\mathbf{Y}$.

Equation (3.5) can also be written as

$$p_{\mathbf{X}}(\mathbf{y}, \mathbf{z}) = p_{\mathbf{Y}|\mathbf{Z}}(\mathbf{y}|\mathbf{z}) p_{\mathbf{Z}}(\mathbf{z}), \quad (3.6)$$

telling us that the joint distribution over $\mathbf{Y}$ and $\mathbf{Z}$ is given as the product of the marginal distribution over $\mathbf{Z}$ and the conditional distribution over $\mathbf{Y}$ given $\mathbf{Z}$. This essentially represents a “reasoning step-by-step” pattern: in order to reason about *both* $\mathbf{Y}$ and $\mathbf{Z}$, first reason about

only $\mathbf{Z}$ and subsequently about $\mathbf{Y}$ given $\mathbf{Z}$. Since the joint distribution is given as a product of marginal and conditional distributions, Equation (3.6) is also referred to as the **product rule**.

There are some technical subtleties with conditioning, as in order for (3.5) to be well-defined, the denominator $p_{\mathbf{Z}}(\mathbf{z})$ needs to be non-zero. When $\mathbf{Z}$ is discrete, however, we might just assume an arbitrary conditional distribution $p_{\mathbf{Y}|\mathbf{Z}}(\cdot|\mathbf{z})$ whenever $p_{\mathbf{Z}}(\mathbf{z}) = 0$. To see this note that for any $A \subseteq \mathcal{Y}$ we have

$$\int_A p_{\mathbf{Y},\mathbf{Z}}(\mathbf{y},\mathbf{z}) d\mathbf{y} \leq \int_{\mathcal{Y}} p_{\mathbf{Y},\mathbf{Z}}(\mathbf{y},\mathbf{z}) d\mathbf{y} = p_{\mathbf{Z}}(\mathbf{z}) = 0.$$

Thus, for any arbitrary $p_{\mathbf{Y}|\mathbf{Z}}(\cdot|\mathbf{z})$ we get

$$\int_A p_{\mathbf{Y},\mathbf{Z}}(\mathbf{y},\mathbf{z}) d\mathbf{y} = \int_A p_{\mathbf{Y}|\mathbf{Z}}(\mathbf{y}|\mathbf{z}) p(\mathbf{z}) d\mathbf{y} = p(\mathbf{z}) \overbrace{\int_A p_{\mathbf{Y}|\mathbf{Z}}(\mathbf{y}|\mathbf{z}) d\mathbf{y}}^{\leq 1} = 0,$$

as required.

When $\mathbf{Z}$ is continuous, and hence $p_{\mathbf{Z}}$ is a density, Equation (3.5) yields a conditional distribution function which is conditional on an event of probability 0. Yet, conditioning is well-defined when the marginal density $p_{\mathbf{Z}}(\mathbf{z})$ is non-zero and continuous at $\mathbf{z}$.

Conditioning is, like marginalization, a consistent reasoning operation, meaning that the order of conditioning operations does not matter. For example, when we start with a joint $p_{\mathbf{X},\mathbf{Y},\mathbf{Z}}$ and want to compute $p_{\mathbf{X}|\mathbf{Y},\mathbf{Z}}$, we might first condition on $\mathbf{Y}$, yielding $p_{\mathbf{X},\mathbf{Z}|\mathbf{Y}}$, and subsequently on $\mathbf{Z}$, or, we condition first on $\mathbf{Z}$ and subsequently on $\mathbf{Y}$. To see this, note that the conditional

$$p_{\mathbf{X},\mathbf{Z}|\mathbf{Y}}(\mathbf{x},\mathbf{z}|\mathbf{y}) = \frac{p_{\mathbf{X},\mathbf{Y},\mathbf{Z}}(\mathbf{x},\mathbf{y},\mathbf{z})}{p_{\mathbf{Y}}(\mathbf{y})},$$

specifies a family of joint distribution over $\mathbf{X}$ and $\mathbf{Z}$ for every value $\mathbf{y}$. Conditioning each of these conditional distributions additionally on $\mathbf{Z}$ delivers

$$\frac{p_{\mathbf{X},\mathbf{Z}|\mathbf{Y}}(\mathbf{x},\mathbf{z}|\mathbf{y})}{p_{\mathbf{Z}|\mathbf{Y}}(\mathbf{z}|\mathbf{y})} = \frac{p_{\mathbf{X},\mathbf{Y},\mathbf{Z}}(\mathbf{x},\mathbf{y},\mathbf{z})}{p_{\mathbf{Y}}(\mathbf{y})} \overbrace{\frac{1/p_{\mathbf{Z}|\mathbf{Y}}(\mathbf{z}|\mathbf{y})}{\int p_{\mathbf{X},\mathbf{Y},\mathbf{Z}}(\mathbf{x},\mathbf{y},\mathbf{z}) d\mathbf{x}}}^{1/p_{\mathbf{Z}|\mathbf{Y}}(\mathbf{z}|\mathbf{y})} = \frac{p_{\mathbf{X},\mathbf{Y},\mathbf{Z}}(\mathbf{x},\mathbf{y},\mathbf{z})}{p_{\mathbf{Y},\mathbf{Z}}(\mathbf{y},\mathbf{z})} = p_{\mathbf{X}|\mathbf{Y},\mathbf{Z}}(\mathbf{x}|\mathbf{y},\mathbf{z})$$

Similarly, by swapping the role of $\mathbf{Y}$ and $\mathbf{Z}$, we will get the same result by first condition on $\mathbf{Z}$ and then on $\mathbf{Y}$.

Conditioning and marginalization are also compatible with each other. For example, when we start with a joint $p_{\mathbf{X},\mathbf{Y},\mathbf{Z}}$ and want to compute $p_{\mathbf{X}|\mathbf{Z}}$, we might

- first condition on $\mathbf{Z}$ (yielding $p_{\mathbf{X},\mathbf{Y}|\mathbf{Z}}$) and then marginalize $\mathbf{Y}$, or

- we might first marginalize $\mathbf{Y}$ (yielding $p_{\mathbf{X},\mathbf{Z}}$) and then condition on $\mathbf{Z}$.

In general, any two sequences of marginalization and conditioning operations leading to the same marginal and/or conditional will deliver consistent results.²

Hence, with the sum and product rule we have established the central “inference machinery” of probabilistic reasoning: we start with a joint distribution $p_{\mathbf{X}}$ describing our domain of discourse. Then we select *variables we want to infer* $\mathbf{Y} \subseteq \mathbf{X}$, depict *variables we know* $\mathbf{Z} \subseteq \mathbf{X}$, and *ignore the rest* $\mathbf{X} \setminus (\mathbf{Y} \cup \mathbf{Z})$. It is worth to embrace the simplicity and elegance of this calculus, allowing us to (repeatedly) apply just two rules to transform our knowledge base $p_{\mathbf{X}}$ into the object of desire, $p_{\mathbf{Y}|\mathbf{Z}}$.

Before we proceed let us simplify a bit our notation. In particular, we have now introduced symbols like $p_{\mathbf{X}}$ and $p_{\mathbf{Y},\mathbf{Z}}$ for joint distributions and symbols like $p_{\mathbf{Z}|\mathbf{X}}$ and $p_{\mathbf{Y},\mathbf{X}|\mathbf{Z}}$ for conditional distributions. Distinguishing these different marginals and conditionals is essential for our intelligent calculus, but using these subscripts quickly becomes cluttered when, for example, describing the product rule:

$$p_{\mathbf{X}}(\mathbf{y}, \mathbf{z}) = p_{\mathbf{Y}|\mathbf{Z}}(\mathbf{y}|\mathbf{z}) p_{\mathbf{Z}}(\mathbf{z}).$$

These subscripts are redundant, since it is clear from the arguments to which random variables the distribution belongs, and also whether it is a conditional or an unconditional distribution. We might just write more compactly

$$p(\mathbf{y}, \mathbf{z}) = p(\mathbf{y}|\mathbf{z}) p(\mathbf{z}).$$

Hence, we will from now omit subscripts of distribution functions when their nature is clear from the arguments.

3.3 The Chain Rule

As mentioned above, the product rule essentially implements a *two-step* reasoning pattern. By applying the product rule repeatedly, we arrive at the *chain rule*, which implements a *multi-step* reasoning pattern. Let $\mathbf{X}$ be a set of random variables which is partitioned into sets $\mathbf{X}_1, \dots, \mathbf{X}_K$. In the following, note that it does not matter whether any $\mathbf{X}_i$, $i \in \{1, \dots, K\}$, contains only a single or several random variables, i.e. the chain rule applies regardless

²This insight is referring to the exact mathematical definition of marginals and conditionals. When performing *approximate* inference the results might of course depend strongly on the order of operations due to numerical and other reasons. In fact, a lot of interesting methods in probabilistic inference is based on clever choices of picking such order and beneficial computational techniques.

whether we consider single random variables or “blocks” of random variables. Using the product rule *repeatedly*, we can now write $p_{\mathbf{X}}$ as

$$\begin{aligned}
 p(\mathbf{x}) &= p(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4, \dots, \mathbf{x}_K) \\
 &= p(\mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4, \dots, \mathbf{x}_K | \mathbf{x}_1) p(\mathbf{x}_1) \\
 &= p(\mathbf{x}_3, \mathbf{x}_4, \dots, \mathbf{x}_K | \mathbf{x}_1, \mathbf{x}_2) p(\mathbf{x}_2 | \mathbf{x}_1) p(\mathbf{x}_1) \\
 &= p(\mathbf{x}_4, \dots, \mathbf{x}_K | \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3) p(\mathbf{x}_3 | \mathbf{x}_1, \mathbf{x}_2) p(\mathbf{x}_2 | \mathbf{x}_1) p(\mathbf{x}_1) \\
 &\quad \dots \\
 &= p(\mathbf{x}_K | \mathbf{x}_1, \dots, \mathbf{x}_{K-1}) p(\mathbf{x}_{K-1} | \mathbf{x}_1, \dots, \mathbf{x}_{K-2}) \dots p(\mathbf{x}_3 | \mathbf{x}_1, \mathbf{x}_2) p(\mathbf{x}_2 | \mathbf{x}_1) p(\mathbf{x}_1) \\
 &= \prod_{k=1}^K p(\mathbf{x}_k | \mathbf{x}_1, \dots, \mathbf{x}_{k-1}). \tag{3.7}
 \end{aligned}$$

In (3.7) the factor $p(\mathbf{x}_k | \mathbf{x}_1, \dots, \mathbf{x}_{k-1})$ for $k = 1$ shall be interpreted as $p(\mathbf{x}_1)$. This factorization of the joint $p_{\mathbf{X}}$ is known as the *chain rule* and represents “step-by-step reasoning” over multiple steps, as it tells us that the joint distribution over $\mathbf{X}_1, \dots, \mathbf{X}_K$ is constructed as the marginal over $\mathbf{X}_1$ times the conditional over $\mathbf{X}_2$ given $\mathbf{X}_1$ times the conditional $\mathbf{X}_3$ given $\mathbf{X}_1, \mathbf{X}_2$, etc.

We are not confined to use the chain rule using the ordering $1, 2, \dots, K$, but are free to use any ordering we like. Specifically, for any permutation $i_1, \dots, i_K$ of the integers $\{1, \dots, K\}$, we can write $p_{\mathbf{X}}$ as

$$p(\mathbf{x}) = \prod_{k=1}^K p(\mathbf{x}_{i_k} | \mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_{k-1}}). \tag{3.8}$$

For example, we might factorize a joint distribution over $\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3$ in various ways:

$$\begin{aligned}
 p(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3) &= p(\mathbf{x}_3 | \mathbf{x}_2, \mathbf{x}_1) p(\mathbf{x}_1 | \mathbf{x}_2) p(\mathbf{x}_2) \\
 &= p(\mathbf{x}_2 | \mathbf{x}_1, \mathbf{x}_3) p(\mathbf{x}_3 | \mathbf{x}_1) p(\mathbf{x}_1) \\
 &= p(\mathbf{x}_1 | \mathbf{x}_3, \mathbf{x}_2) p(\mathbf{x}_2 | \mathbf{x}_3) p(\mathbf{x}_3) \\
 &\quad \text{etc.}
 \end{aligned}$$

Thus, rather than a single chain rule, for a given partitioning $\mathbf{X}_1, \dots, \mathbf{X}_K$ there really exist $K!$ different chain rules. The chain rule allows to factor and analyse joint distributions in a flexible way and can be considered as a “Swiss army knife” of probabilistic inference.

Conversely, reading the chain rule in the other direction, we can *construct* expressive joint distributions by providing a marginal or conditional for each $\mathbf{X}_k$ and multiplying these

together. Here, the conditionals need to obey *some* ordering of the $\mathbf{X}_k$'s, meaning that an $\mathbf{X}_k$ can only appear as conditioned in some conditional distribution $p_{\mathbf{X}_l|\dots,\mathbf{X}_k,\dots}$ when $\mathbf{X}_k$ comes before $\mathbf{X}_l$ in this order. As long as there exists such an order, the chain rule leads to a proper joint distribution $p_{\mathbf{X}}$. This is the main working principle both behind classical Bayesian networks [28] and autoregressive distribution estimators [29].

3.4 Bayes Rule (Law of Inverse Probability)

Another consequence of the product rule is *Bayes rule* or the *rule of inverse probability*, which has been discussed already in Section 2.4.3 on the level of probability measures. When considering two (sets of) random variables $\mathbf{X}, \mathbf{Y}$, we might write the joint as

$$p(\mathbf{x}, \mathbf{y}) = p(\mathbf{y}|\mathbf{x}) p(\mathbf{x})$$

or also

$$p(\mathbf{x}, \mathbf{y}) = p(\mathbf{x}|\mathbf{y}) p(\mathbf{y}).$$

By equating the right hand sides and dividing by $p(\mathbf{x})$ we yield Bayes rule

$$p(\mathbf{y}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{y}) p(\mathbf{y})}{p(\mathbf{x})} = \frac{p(\mathbf{x}|\mathbf{y}) p(\mathbf{y})}{\int p(\mathbf{x}|\mathbf{y}) p(\mathbf{y}) d\mathbf{y}}, \quad (3.9)$$

which allows us to change the “direction of reasoning” from $\mathbf{x} \leftarrow \mathbf{y}$ to $\mathbf{y} \leftarrow \mathbf{x}$. Bayes rule is in particular prominent in applications where $\mathbf{Y}$ is some unobserved (latent) quantity, such as an unknown model parameter, the truth status (*true, false*) of a hypothesis, or some other (assumed) latent structure underlying an application, while $\mathbf{X}$ is an observed quantity (data). In such settings, it is often natural to describe how the data $\mathbf{X}$ emerges from the latent quantity $\mathbf{Y}$ via some conditional $p_{\mathbf{X}|\mathbf{Y}}$, usually called the *likelihood*. Together with a marginal distribution $p_{\mathbf{Y}}$, usually called the *prior*, Bayes rule (3.9) allows us to infer the conditional $p_{\mathbf{Y}|\mathbf{X}}$ of interest, called the *posterior*. In this way, Bayes rule captures a wide range of inverse problems in science and engineering.

3.5 (Conditional) Independence

The concept of independence discussed in Section 2.4.4 naturally carries over to distributions functions. The conditional distribution $p_{\mathbf{Y}|\mathbf{X}}$ represents our information about $\mathbf{Y}$ when we have learned the value for $\mathbf{X}$. If this is the same as when we do *not* know the value of $\mathbf{X}$,

i.e. the marginal p_Y , we conclude that Y and X are independent. Consequently, we define $X \perp\!\!\!\perp Y$, when the following equivalent conditions hold for all values x, y :

- $p(x|y) = p(x)$
- $p(y|x) = p(y)$
- $p(x, y) = p(x)p(y)$

Independence can occur or disappear in the context of another set of random variables Z . In particular, we say that X and Y are *conditionally independent* given Z , written $X \perp\!\!\!\perp Y | Z$, if the following equivalent conditions hold for all values x, y, z :

- $p(x|y, z) = p(x|z)$
- $p(y|x, z) = p(y|z)$
- $p(x, y|z) = p(x|z)p(y|z)$

3.6 Expectations

So far, we have established that probability allows us to *represent knowledge in the form of a joint distribution* and provides two simple inference rules to process our knowledge, *the sum and the product rule*. We have also established the chain rule and Bayes rule, which, however, are really just variants of the product rule. All this calculus does is processing distributions into other distributions, as illustrated in the following example.

Example 4. Assume we have access to the joint distribution over 21 random variables $Y, X_1, \dots, X_{20}$, where Y is some medical parameter of interest (e.g. cerebral blood flow), and $X_1, \dots, X_{20}$ are some measurements related to Y (e.g. features derived from 20 EEG channels). Further, assume that for a particular patient we have measured (observed) $X_1 = x_1, X_2 = x_2, \dots$, but X_{17} and X_{19} have not been measured (because for these two the corresponding EEG sensors have dropped out). Hence, employing the sum and product rule we compute

$$p(y|x_1, \dots, x_{16}, x_{18}, x_{20}) = \frac{\int \int p(y, x_1, \dots, x_{16}, x_{17}, x_{18}, x_{19}, x_{20}) dx_{17} dx_{19}}{\int \int \int p(y, x_1, \dots, x_{16}, x_{17}, x_{18}, x_{19}, x_{20}) dx_{17} dx_{19} dy}, \quad (3.10)$$

which is our inferred distribution about Y , incorporating all available information.

In Example 4, one might wonder what shall be done next with $p_{Y|X_1, \dots, X_{16}, X_{18}, X_{20}}$, a potentially very complicated and multimodal distribution?³ How do we get a *concrete* prediction for Y ? The first response to these concerns is that *nothing* should be done, since the conditional distribution over Y is our concrete prediction. More precisely, *if* (i) the joint $p_{Y, X_1, \dots, X_{20}}$ is indeed the true distribution over all involved quantities, or at least a sufficiently good approximation of it, (ii) the observed values $x_1, \dots, x_{16}, x_{18}, x_{20}$ are indeed all the information we have⁴ and (iii) (3.10) is solved exactly, there is nothing more that we can do. The conditional $p_{Y|X_1, \dots, X_{16}, X_{18}, X_{20}}$ captures everything we can know about Y based on the available information, or actually, it captures what we *don't* know, namely our uncertainty about Y . If the distribution is unimodal and very concentrated we are pretty confident about the value of Y , while if it is spread out and multimodal we are pretty uncertain. Keeping the whole distribution of Y means that *we know what we don't know*.

However, humans tend to be uncomfortable with this situation. They want to call the shots and know a concrete single value for Y . Often, committing to a single value is also an external requirement, forcing us to make a *decision*. The basic tool to do so is **expectation**, which is simply the average value a random variable assumes:

$$\mathbb{E}[X] := \int p(x)x \, dx, \quad (3.11)$$

where the integral is taken over the state space of X . For discrete random variables, the integral is replaced by a sum. For random vectors the expectation is a vector-valued quantity:

$$\mathbb{E}[\mathbf{X}] := \int p(\mathbf{x})\mathbf{x} \, d\mathbf{x}. \quad (3.12)$$

Expectations can be naturally applied to conditional distributions. When $p_{Y|Z}$ is a conditional distribution, the conditional expectation of Y given the event $Z = \mathbf{z}$ is

$$\mathbb{E}[Y | \mathbf{z}] = \int p(y | \mathbf{z})y \, dy. \quad (3.13)$$

In Example 4, computing $\mathbb{E}[Y | x_1, \dots, x_{16}, x_{18}, x_{20}]$ is one particular way to decide a value for Y . However, being an average does not necessarily make it a typical value of the distribution, as illustrated in Example 5.

³The *mode* of a distribution is the value of y for which $p(y)$ is maximal. A “bumpy” distribution function which has more than one (local) maxima is called *multimodal*. Intuitively, multimodality signals ambiguity about Y .

⁴Note that here we assume that X_{17} and X_{19} are *missing at random*, meaning that the fact that they are missing is independent of Y given the observed RVs [52]. In this case (3.10) is an exact conditional of Y given the available data. If they were not missing at random, one would need to account for this fact, for example by explicitly incorporating the missingness process in the model.

Example 5 (Expectation of a Bernoulli). *Consider a Bernoulli distribution over a random variable X with state space $\mathcal{X} = \{0, 1\}$, assigning probabilities $p_X(1) = \theta$ and $p_X(0) = 1 - \theta$, where θ is a single parameter called the success probability. When $\theta = 0.5$, the expectation is $\mathbb{E}[X] = 0.5$ which is a value never assumed by X . Generally, one can easily construct multimodal distributions for which the expectation might lie somewhere between the modes in an arbitrary large area of 0 probability.*

As the expectation is describing some aspect of the distribution over $\mathbf{X}$, an immediate idea is to also consider expectations of transformations of $\mathbf{X}$. In particular, for some (measurable) function g and we might define $\mathbf{Y} = g(\mathbf{X})$. Then, the expectation of $\mathbb{E}[\mathbf{Y}]$ will also be describing aspects of the original distribution $p_{\mathbf{X}}$. Computing $\mathbb{E}[\mathbf{Y}]$ in principle requires that we first figure out $p_{\mathbf{Y}}$, which is often a delicate task. However, there is another and often simpler way, as it holds that

$$\mathbb{E}[g(\mathbf{X})] = \mathbb{E}[\mathbf{Y}] = \int p(\mathbf{y})\mathbf{y}d\mathbf{y} = \int p(\mathbf{x})g(\mathbf{x})d\mathbf{x}. \quad (3.14)$$

This identity is known as the *law of the unconscious statistician (LOTUS)*, as the right hand side is often considered as a definition of $\mathbb{E}[g(\mathbf{X})]$, even though it is actually a theorem.

For univariate random variables, a common choice for g are powers, i.e. $g(x) = x^K$, where $k \in \mathbb{N}$, and the expectation $\mathbb{E}[X^k]$ is called the k^{th} *moment* of p_X . The first moment is of course just the expectation $\mathbb{E}[X]$ also called the *mean* of p_X . The second moment of a transformed version of X , centered at 0 by subtracting its mean, is the *variance*

$$\mathbb{V}[X] := \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2. \quad (3.15)$$

For random vectors $\mathbf{X}$, the corresponding concept is the *covariance matrix*:

$$\text{Cov}[\mathbf{X}] := \mathbb{E}[(\mathbf{X} - \mathbb{E}[\mathbf{X}])^T (\mathbf{X} - \mathbb{E}[\mathbf{X}])] = \mathbb{E}[\mathbf{X}^T \mathbf{X}] - \mathbb{E}[\mathbf{X}]^T \mathbb{E}[\mathbf{X}]. \quad (3.16)$$

The diagonal of Cov are the variances of the individual random variables in $\mathbf{X}$, while the off-diagonal contains the covariances between any pair $X, Y \in \mathbf{X}, X \neq Y$:

$$\text{Cov}[X, Y] = \mathbb{E}[(X - \mathbb{E}[X]) (Y - \mathbb{E}[Y])]. \quad (3.17)$$

In summary, expectations are one of the main tool to convert probability distributions into “concrete values”. This concept becomes in particular important when paired with optimization (taking maxima or minima), leading to *decision making*, see Section 3.8.

3.7 Most Probable Explanation

Rather than averaging we might also take the extreme values, i.e. maxima or minima. In particular, when p is a joint distribution over random variables $\mathbf{X} = (X_1, \dots, X_D)$ with state space $\mathcal{X}$, then we might ask which state is the most likely one, i.e. the state with maximal probability:

$$\mathbf{x}^* = \arg \max_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \quad (3.18)$$

Here we assume the maximum is unique. If it is not, we might either break ties in some way or return the set of all maxima. The value $\mathbf{x}^*$ is known as *most probable explanation (MPE)* [39].

The MPE is particularly interesting for conditional distributions. Referring to Example 4, the MPE for Y given the observed information would be

$$y^* = \arg \max_y p(y | x_1, \dots, x_{16}, x_{18}, x_{20}) \quad (3.19)$$

If Y is a discrete random variable, and hence the conditional distribution function is a PMF, y^* is indeed the most probable value for Y for the given evidence.

When Y is continuous, and hence the conditional is a density, the interpretation of y^* is a bit more subtle, as illustrated in Example 6. The MPE in this example is perhaps surprising, as the value $Y = 9$ maximizes the density $p_Y(9) \approx 0.8$, but is located in a region attributing only about 0.1% of the total probability mass. If we were to pick the other mode around $Y = -10$, we would get a smaller value for $p_Y(-10) \approx 0.2$ but be in a region accounting for 99.9% of the probability.

Is MPE flawed for densities? No, the seeming paradox comes from making tacit assumptions. MPE makes the following promise: assuming that p_Y is continuous, then there exists some $\varepsilon > 0$ such that the probability that Y falls into the interval $[y^* - \varepsilon, y^* + \varepsilon]$ is maximal when y^* is the MPE. In Example 6, if we discretize the state space of Y into very small cells and bet on the outcome in which cell Y will land, we are best off by committing to the MPE at $y^* = 9$ (for small enough cell size). This might often not be what we want for real-valued state spaces. The solution to such apparent paradoxes is offered by decision theory.

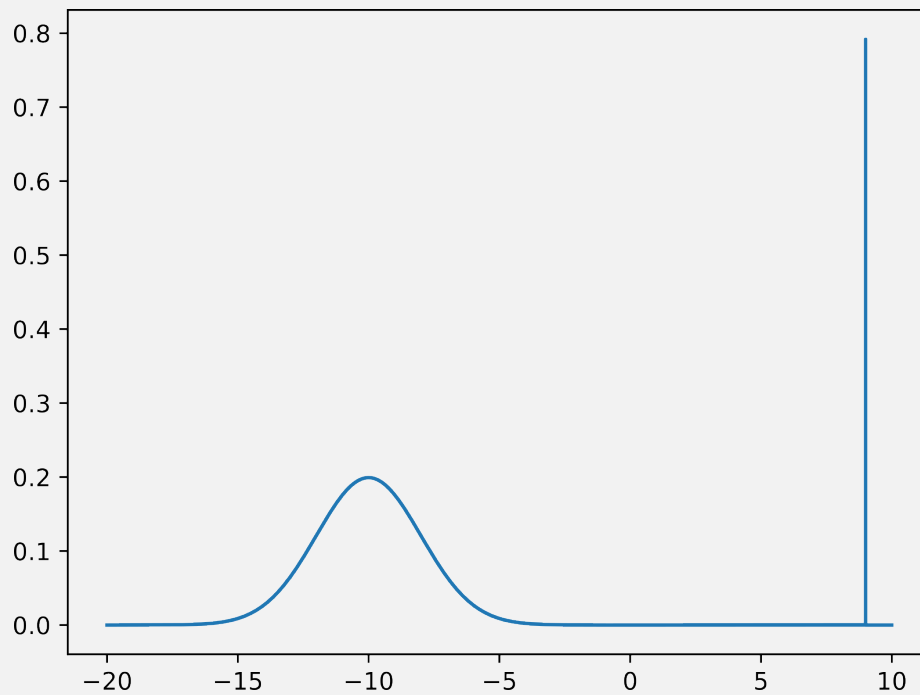
3.8 Decision Theory

The issue with Example 6 is that many would prefer the mode of the left Gaussian, since with 99.9% probability Y takes a value in the vicinity of -10 . Intuitively, if we guess $Y = -10$ but the true value turns out to be, say, $Y = -10.13$, we still would be happy, since we have

Example 6 (MPE with Densities). Assume that Y has the following density

$$p_Y(y) = 0.999p_N(y | -10, 2) + 0.001p_N(y | 9, 0.0005),$$

where $p_N(y | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y-\mu)^2}{2\sigma^2}}$ is the Gaussian density. Hence, $p(y)$ is a Gaussian mixture with two components, one of which represents 0.999 of the total probability mass and the other 0.001. This density is shown in the following figure:



Here we omit any conditioning information, but we might figure p_Y as a (toy) distribution resulting from (3.10) in Example 4. The MPE in this example, i.e. the state where p_Y is maximal, is $y^* = 9$, or actually slightly less, due to the Gaussian component on the left.

guessed a value *close* to the true value. As illustrated above, MPE essentially discretizes the space into tiny cells and deliberately ignores any notion of *closeness* among the cells—if we bet on the neighboring cell of where Y really lands, “we still lose” according to MPE. Decision theory [4] allows us to make such assumptions explicit, by introducing a *loss function* and a simple recipe for making optimal decisions under uncertainty.

Assume we aim to decide on the value of a scalar quantity of interest Y (the case for random vectors is straightforward), and let $\mathcal{Y}$ be the state space of Y . In order to make this decision we assume that we have evidence $\mathbf{X} = \mathbf{x}$, and that we have already computed the conditional $p_{Y|\mathbf{X}}$, by applying the sum and product rule. If no evidence is available, we would use the marginal p_Y instead. In either case, we arrive at the required distribution by mere application of the sum and product rules.

A *loss function* ℓ is any function

$$\ell: \mathcal{Y} \times \mathcal{Y} \mapsto \mathbb{R}. \quad (3.20)$$

Given $y^*, y_t \in \mathcal{Y}$, the value $\ell(y^*, y_t)$ represents our regret when we decide for value y^* , but actually $Y = y_t$. Usually, loss functions will satisfy $\ell(y_t, y_t) \leq \ell(y^*, y_t)$ for all $y^* \in \mathcal{Y}$, that is, our loss will be minimal if we correctly guess the true value of Y .

Now, the basic rule of decision theory is to decide for the value which minimizes the *expected loss*, conditional on the available evidence $\mathbf{X} = \mathbf{x}$:

$$y^* = \arg \min_{y \in \mathcal{Y}} \mathbb{E}[\ell(y, Y) | \mathbf{x}]. \quad (3.21)$$

The frequentist interpretation of probability gives a clear argument why this is an optimal decision rule: in a scenario of making repeated decisions, rule 3.21 will lead to the minimal sum of losses if we correctly inferred the true $p_{Y|\mathbf{X}}$ and the number of decision trials goes to infinity. Let us study a few basic loss functions in the context of Example 6.

Zero-one loss. Consider for some $\varepsilon > 0$ the loss function

$$\ell_{0/1}(y^*, y) = \begin{cases} 0 & \text{if } |y^* - y| < \varepsilon \\ 1 & \text{otherwise,} \end{cases}$$

denoted as *zero-one* loss. If the (conditional) density of Y is continuous, then (3.21) under zero-one loss approaches the MPE when $\varepsilon \rightarrow 0$. Hence MPE essentially minimizes zero-one loss.

Squared distance loss (ℓ_2 -loss). The “all-or-nothing” behavior of MPE is probably not what one wants in Example 6, as it delivers an “atypical” value in an area of low probability. Rather, one would probably prefer the *squared distance loss*

$$\ell_2(y^*, y) = (y^* - y)^2,$$

as it assumes that we are still happy if we guess a values close to the true value of Y . It can be shown that for any distribution p_Y , the *least squares loss*, i.e. (3.21) when using ℓ_2 , is achieved by the mean of p_Y . For a Gaussian mixture model the mean is given as the corresponding mixture of the components’ means. Specifically, in Example 6, we get the solution

$$y^* = 0.999 \times (-10) + 0.001 \times 9 = -9.981, \quad (3.22)$$

delivering a value almost at the center of the high probability region on the left.

Absolute distance loss (ℓ_1 -loss). However, one might be concerned by the fact that the least squares solution (3.22) is quite influenced by the locations of the Gaussian components. In particular, if the mean of the right Gaussian component (the one with weight 0.001) was moved to a very large value, e.g. 9000, we would get $y^* = -0.99$ which is far away from the high probability area. This proneness to outliers is typical for least squares. A remedy to this problem is to use *absolute distance loss* defined as

$$\ell_1(y^*, y) = |y^* - y|.$$

It can be shown that the minimum of (3.21) when using ℓ_1 is achieved by the *median* of p_Y , which is the value y^* for which $\mathbb{P}(Y \leq y^*) = \mathbb{P}(Y \geq y^*) = 0.5$. It can be figured out that the median in Example 6 is about -9.9975 and it stays virtually the same if we move the mean of the right Gaussian to 9000 or even far more extreme values. This robustness to outliers is a well known property of ℓ_1 -loss and the median.

In summary, minimizing expected loss (3.21) is a principled approach to “call the shots” and transit from our probabilistic prediction—a conditional distribution over the quantity of interest Y —to a concrete hard value y^* . It should be advised that—in the ideal setting—this step should come very last, since it discards all the beautiful “fluffy uncertainty” about Y and approximates it with a brutally hard single value.

In particular in Example 4, figure the scenario that inferring the cerebral blood flow Y was only an intermediate step, and that the ultimate goal is to infer, together with other medical parameters, the health condition of a patient. In this scenario it is crucial to keep the entire

$p(y | \dots)$ and process it further downstream. Approximating it in any way before we have arrived at the final inference goal compromises the exact reasoning calculus of probability, destroys valuable information, and can—in principle—deliver arbitrary bad results. In other words, we shall “stay calm” and continue applying the sum and product rule.⁵

Some points are worth mentioning. First, defining a loss function ℓ and applying decision rule (3.21) makes our assumptions *explicit* and allows us to specify about what we care when deciding a value y^* . However, designing a loss function for a concrete application is not trivial. For example, when we are dealing with a *binary classification* problem, i.e. $Y \in \{0, 1\}$, where $Y = 0$ represents the absence of a deadly disease and $Y = 1$ represents its presence, it is quite delicate to come up with good loss function. It is clear, that predicting $y^* = 0$ (patient healthy) when in fact $Y = 1$ (patient ill) should incur a higher loss than for the converse case. But, how much higher? We might fix an arbitrary price, e.g. 1 Euro, to diagnosing the patient as ill when in fact they are healthy, but how much does it cost when we release an actually ill patient and endanger their life? An ethically delicate question.

Yet, I want to stress again that *if* (i) the loss function ℓ indeed adequately represents our preferences, (ii) we have access to the true data generating distribution $p_{Y,X}$ and (iii) we can actually compute (3.21) *exactly*, there is nothing left to do—we have successfully made an optimal decision in terms of expected loss. There are a lot of “if’s” here and I am not saying that these computations are easy, yet it is surely beneficial to know and specify what is the best one can do. For example, in the case of classification, the MPE computed for the true data distribution leads to the *Bayes optimal classifier*, i.e. the classifier with minimal zero-one loss. A famous result for the *K-nearest neighbor classifier* is that it asymptotically converges to the Bayes optimal classifier (given that the number of neighbors grows also to infinity, but less strongly than the number of data points), making it a theoretically well-justified classification rule [7]. In the context of regression, using the data generating distribution and ℓ_2 loss leads to the *optimal regression function*, and no ever-so-fancy machine learning method will be able to outperform it in terms of true squared loss.

Finally, it is worth mentioning that (3.21) is not the only good decision rule. For example, we might not care about the *expected loss* too much but rather predict a value such that the suffered loss is smaller than a certain threshold with a prescribed probability, and otherwise reject the decision, leading to a more risk aware prediction scheme. Moreover, *conformal prediction* [54] gives up on the property to return only a single value but rather returns a

⁵A typical criticism raised at this point is that probabilistic modeling and inference are hard, and that all the required approximations along the way might be much worse than replacing intermediate distributions with concrete values. This is certainly a valid point, but conflates two aspects: one aspect, which is the central one in this thesis, is asking what **in principle** should be done, i.e. what is in principle an optimal calculus of reasoning under uncertainty? The other aspect is what can be done in reality and how well our optimal calculus can actually be implemented.

minimal set of predictions $\mathcal{Y}^* \subset \mathcal{Y}$ such that the true value of Y ends up in $\mathcal{Y}^*$ with high probability.

Chapter 4

State of The Art

In this chapter, I provide an overview of state-of-the-art probabilistic approaches in AI. While AI was largely dominated by logic-based methods until the late 1980s, the necessity of accounting for uncertainty in real-world scenarios became increasingly recognized. Although probability was not immediately embraced—it actually faced quite some criticism for its computational challenges and perceived impracticality—it gradually gained ground. A key proponent was Judea Pearl, whose seminal book *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference* [39] firmly established probabilistic reasoning as a central approach in AI.

Graphical Models, in particular Bayesian networks, were central to Pearl’s work and have since remained a cornerstone of probabilistic modeling [28]. Although their prominence has somewhat diminished in the deep learning era, they still serve as a *lingua franca* of probabilistic modeling, due to their intuitive visual representation, which correspond directly with conditional independence assumptions and tractability in inference and learning [39, 28].

Inference in graphical models—especially marginalization via the sum rule—is generally NP-hard [11]. However, exact inference is tractable for models with bounded *treewidth*, a measure of how tree-like a graph is. Most probabilistic machine learning systems can be mapped onto a corresponding graphical model, and authors often use such them to concisely express model structure and assumptions.

Deep Autoregressive Models [29, 70] apply the chain rule of probability and model each conditional distribution using a shared neural network. These models can be interpreted as fully connected Bayesian networks, which are capable, in principle, of representing any joint distribution. While they support sampling and density evaluation, computing marginals, conditionals, or performing complex inference tasks remains challenging.

Variational Autoencoders (VAEs) [26] are a neural latent variable model, previously known as *density models* [33]. They introduce a latent code with a simple distribution, often isotropic Gaussian, which is transformed by a neural network to produce the (parameters of) a conditional distribution over observable variables; hence they represent an infinite mixture model, with the latent variable marginalized out.

Exact inference in VAEs (density networks) is a hard task, but the *evidence lower bound* (ELBO), that is, a lower bound on the log-likelihood can be estimated. A major innovation of VAEs was the introduction of *amortized inference*: instead of computing the posterior for each data point individually, a separate network—commonly called the *encoder* or *inference network*—learns to approximate it. The encoder is used to compute an ELBO estimate which used as the VAEs learning objective as a replacement for the exact log-likelihood. By optimizing both the parameters of the model and the encoder, one achieves the dual goal to improve the ELBO estimate and optimizing the model parameters at the same time.

Generative Adversarial Networks (GANs) [22] also introduce a simple latent distribution but adopt an adversarial training scheme rather than maximizing a log-likelihood bound. They involve a second network—the *discriminator* or *critic*—which learns to distinguish real data from generated samples. This training objective corresponds to minimizing a probabilistic divergence distinct from maximum likelihood [22, 3].

While GANs enable realistic sampling, they generally do not allow density evaluation (often such densities do not exist in the Lebesgue sense), nor do they support tractable inference for marginals or conditionals.

Normalizing Flows [49, 37] are another principle to probabilistic modeling. Similar as VAEs and GANs, they start with a simple distribution (e.g., an isotropic Gaussian) and apply a series of bijective, differentiable transformations to generate samples from a more complex target distribution. The exact density of the transformed distribution is computed via the change of variables formula, and exact sampling is also feasible. However, computing marginals or conditionals remains challenging, similar to VAEs and GANs.

Energy-Based Models define distributions using an unnormalized energy function $E(\mathbf{x})$, often parameterized by a neural network:

$$p(\mathbf{x}) = \frac{\exp(-E(\mathbf{x}))}{\int \exp(-E(\mathbf{x})) d\mathbf{x}}.$$

While extremely flexible, these models are typically intractable. Evaluating densities, drawing samples, and performing inference are all computationally expensive. Marginals and conditionals are especially difficult to estimate reliably.

Score-Based Models [77] are closely related to energy-based models and represent a distribution via its *score function*, i.e., the gradient of the log-density, $\nabla \log p(\mathbf{x})$. A neural network is trained to approximate this score. When combined with diffusion processes and Langevin dynamics, these models currently represent the state of the art in generative modeling for images [61].

Despite their impressive performance, score-based models usually lack tractable density evaluation and require approximations for sampling, marginals, and conditionals. These approximations can be difficult to validate in practice.

Gaussian Processes (GPs) [79] are another prominent type of probabilistic model. Formally, GPs are a (possibly uncountable) collection of random variables, of which any finite subset has a multivariate Gaussian distribution. Usually, these random variables are indexed by some continuous parameter, e.g. $x \in \mathbb{R}$, which allows us to interpret GPs as random functions. GPs have ample applications in Bayesian modeling of latent functions and Bayesian optimization [59]. Due to Gaussianity, computing marginals and conditionals in GPs is tractable, but associated with a cubic computational cost in the number of data points, which becomes delicate for large data sets. These challenges are most commonly addressed with *sparse inducing point techniques* [47].

Probabilistic Programming (PP) [69] is a general framework to express probabilistic models in a flexible way, namely by writing a program equipped with random variables. When evaluating such a program in the standard way, it simply amounts to a stochastic simulator. However, the main powerful idea of PP is that such a program constitutes a probabilistic model which can be subject to inference. In this way, one might provide data for the observed output of a probabilistic program and query the posterior distribution over certain latent parameters, in principle inferred via Bayes’ rule. In practice, this inference is tackled via approximation, usually employing variational inference or Monte Carlo techniques. In a nutshell, PP aims to achieve for probabilistic modeling and inference what machine learning frameworks like TensorFlow and PyTorch have achieved for deep learning.

The list of probabilistic modeling approaches above is not complete, but captures a coarse picture of “what is going on in the field”. A remarkable point is that many models mainly

strive for expressivity and neglect the question about inference, or actually, heavily rely on approximation such as sampling and variational inference.¹ This is in quite stark contrast to the picture of probability as a sound and rigorous reasoning framework under uncertainty. Reasons for this neglect are, in my personal opinion, stemming from a particular mindset (i) assuming that tractable models necessarily underperform and (ii) tailored towards the contemporary success of deep learning (i.e. function approximation) style of AI. Probabilistic circuits, as discussed in the following, put an explicit focus on tractability.

¹The most noteworthy exceptions are low-tree width graphical models and GPs.

Chapter 5

Probabilistic Circuits

So far, I have presented probability as an elegant and rigorous inference language for AI, boiling down to two simple rules, the sum and product rule. Then, in order to make (optimal) decisions, all we need to do on top of that is taking expectations, maxima and minima. Of course, this actually requires us that we have access to the *true* underlying data distribution or a reasonably good approximation. If our model defers too much from reality, all the inferences we are doing will be worthless—garbage in, garbage out. Hence, faithful modeling and learning is a crucial part of the game.

Probabilistic models are generally some representation of a distribution function $p_{\mathbf{X}}$, such as *probabilistic graphical models* (PGMs) [28], *density networks* and *variational autoencoders* (DNs, VAEs) [33, 26, 50], *neural auto-regressive distribution estimators* (NADEs) [29], *energy-based models* [60], *score-based models* [77], *normalizing flows* [49], *probabilistic programs* [6], and *Gaussian processes* (GPs) [79].¹ A common model which does not represent a distribution function $p_{\mathbf{X}}$ (yet a distribution $P_{\mathbf{X}}$) is *generative adversarial networks* (GANs) [22], as they usually do not admit a density with respect to Lebesgue measure. All these models represent distributions in various creative and flexible ways, so that we are hopefully able to represent and learn the distribution we are interested in.

Probabilistic inference, however, is generally intractable in most of these models. Exceptions are PGMs with *bounded tree-width* [28] (computing marginals is exponential in the tree-width) and GPs (computing conditionals is cubic in the number of samples). For all other models, computing marginals and/or conditionals is a hard computational problem, as they are all based on high-dimensional non-linear functions (neural networks). Computing integrals—the core business of inference—over such functions is a notoriously hard task.

¹GPs are technically a collection of (infinitely many) random variables with the property that any finite dimensional marginal is Gaussian. Hence, the distribution function of any finite subset of random variables is defined.

Probabilistic circuits (PCs) [73, 9] are a type of **tractable** probabilistic model that allows to perform inference **exactly** and **efficiently**, meaning that the probabilistic inference task of choice (marginals, conditionals, etc.) are computable without approximation² and in polynomial time of the PC’s size. Hence, PCs stay at all times faithful to the probabilistic reasoning principle: when we manage to represent our distribution of interest as a PC, we do have the guarantee that we arrive at the correct conclusion. This property makes PCs quite unique among other probabilistic models, which typically strive for maximal expressivity rather than tractable inference.

Evidently, there must be a catch to PCs. Basically, PCs “buy” tractability by accepting rather stringent constraints on (or properties of) their structure. These constraints make them less expressive than other types of model, meaning that there exist probability distributions which have a compact representation (polynomial in the number of random variables) as, say, a PGM, but an exponential representation size as a PC. These theoretical arguments, which go under the name *expressive efficiency* [35], are however not fully conclusive. In particular, the PC community has steadily been making progress over the last two decades and demonstrated that tractable models often compete well with intractable ones or even outperform them. Some examples are provided in Part II of this thesis.³ Furthermore, it should be noted that PCs are still universal approximators of densities, that is, they absolutely *can* represent any distribution we like. The representation size, however, might just be infeasible.

The term “probabilistic circuits” is a relatively new one, but the principle has been around since decades and is known under many different names. *Arithmetic circuits* [14] are probably the earliest representatives and clearly display the key ideas of PCs and other tractable circuits (operating e.g. in the logic domain). The concept of PCs is also reflected in *AND/OR-graphs* [17], *sum-product networks* [46] and many other models. The main idea to use yet another name, put forward by several researchers at UCLA [9], is to unify these superficially different names and concepts under one umbrella.

In this chapter, I introduce PC and explain their working principles. Furthermore, I give an overview of the work I have done in this area over the last 10 years, which also concludes Part I of this cumulative Habilitation thesis. In Part II, the relevant papers I have published are listed as chapters.

²Leaving aside numerical errors stemming from finite precision arithmetic.

³For more examples, see e.g. our latest tutorial at NeurIPS’22 [72].

5.1 Basic Definitions of Probabilistic Circuits

We start with the formal definition of PCs.

Definition 12 (Probabilistic Circuit (PC)). *Let a collection of random variables $\mathbf{X} = (X_1, \dots, X_D)$ be given, let $\mathcal{X}_1, \dots, \mathcal{X}_D$ be their state spaces and $\mathcal{X} = \times_i \mathcal{X}_i$ the joint state space of $\mathbf{X}$. A probabilistic circuit (PC) is a representation (model) of a potentially unnormalized joint probability distribution defined on $\mathcal{X}$, i.e., a function*

$$p: \mathcal{X} \mapsto [0, \infty]. \quad (5.1)$$

Specifically, a PC is a type of neural network, i.e. a (directed acyclic) computational graph $\mathcal{G} = (V, E)$ containing three types of computational nodes, namely probability distribution functions (short distributions), weighted sums and products. In the following we introduce the symbols $\mathbf{D}$, $\mathbf{S}$ and $\mathbf{P}$ to refer to some generic distribution, sum and product node, respectively, while $\mathbf{N}$ refers to a generic node of any type. Further, let $\mathbf{D}$, $\mathbf{S}$ and $\mathbf{P}$ denote the set of all distributions, sums and products, respectively.

For any $\mathbf{N} \in V$, let $\mathbf{in}(\mathbf{N})$ denote the neighbors of $\mathbf{N}$ with an edge towards $\mathbf{N}$, and $\mathbf{out}(\mathbf{N})$ be the neighbors with an edge outgoing from $\mathbf{N}$. The input to the PC is some state $\mathbf{x} = (x_1, \dots, x_D) \in \mathcal{X}$ for the random variables ($\mathbf{x}$ is not contained in V).

The leaves of $\mathcal{G}$ are the distribution nodes, i.e., $\mathbf{D} = \{\mathbf{N} \in V \mid \mathbf{in}(\mathbf{N}) = \emptyset\}$, while all other nodes are either sum or product nodes. Moreover, only the distribution nodes get $\mathbf{x}$ as direct input (and by construction no input from any $\mathbf{N} \in V$), while sum and product nodes can only get direct input from nodes in V . Thus, the distribution nodes $\mathbf{D}$ form the first layer in the PC, while sums and products are located on “higher” layers. For that reason, distribution nodes are also often called input distributions of the PC.

*A **distribution node** $\mathbf{D}$ is a distribution function over some subset of $\mathbf{X}$. This subset, written as $\mathbf{X}[\mathbf{D}] \subseteq \mathbf{X}$, is denoted as the scope or receptive field of $\mathbf{D}$. Hence, when evaluating the PC for some joint state $\mathbf{x}$, node $\mathbf{D}$ only receives $\mathbf{x}[\mathbf{D}]$, defined as the vector containing the values in $\mathbf{x}$ corresponding to $\mathbf{X}[\mathbf{D}]$. The value computed by $\mathbf{D}$ is the distribution function evaluated at $\mathbf{x}[\mathbf{D}]$, which is technically $\mathbf{D}(\mathbf{x}[\mathbf{D}])$, but usually written simply as $\mathbf{D}(\mathbf{x})$. A distribution node might have trainable parameters, which are denoted as $\boldsymbol{\theta}_{\mathbf{D}}$.*

*A **sum node** $\mathbf{S}$ with real weights $\boldsymbol{\theta}_{\mathbf{S}} = (\theta_{\mathbf{S}\mathbf{N}})_{\mathbf{N} \in \mathbf{in}(\mathbf{S})}$ computes a nonnegative mixture of its inputs, i.e.,*

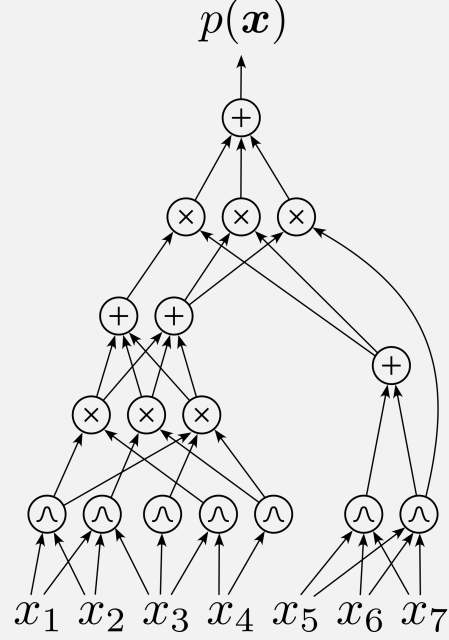
$$\mathbf{S}(\mathbf{x}) = \sum_{\mathbf{N} \in \mathbf{in}(\mathbf{S})} \theta_{\mathbf{S}\mathbf{N}} \mathbf{N}(\mathbf{x}) \quad (5.2)$$

where $\theta_{\mathbf{S}\mathbf{N}}$ are (trainable) weights satisfying $\theta_{\mathbf{S}\mathbf{N}} \geq 0$.

Example 7 (Probabilistic Circuit).

An example PC over seven random variables $X_1, \dots, X_7$ is shown on the right. A complete state (observation) $\mathbf{x} = (x_1, \dots, x_7)$ serves as input to the PC. The first layer consists of distributions nodes (D's), here illustrated with nodes showing a Gaussian-like distribution functions (they are, however, not necessarily Gaussian, but any distribution function can be used). The scope of each distribution node can be recognized by which input values are connected to it.

The nodes with $+$ and $\times$ symbols are the sum and product nodes, respectively. The PC computes a density by evaluating the circuit bottom up, as usual in neural networks. The return value is $p(\mathbf{x})$, the PC distribution evaluated at $\mathbf{x}$. Parameters for sum weights and distribution nodes are not shown in the figure.



A **product node** P computes the product of its inputs, i.e.,

$$P(\mathbf{x}) = \prod_{N \in \text{in}(P)} N(\mathbf{x}). \quad (5.3)$$

Note since distribution nodes have a restricted scope, that is, a subset of $\mathbf{X}$, sum and product nodes will also have a restricted scope. For an arbitrary sum or product node N , its scope is recursively given as

$$\mathbf{X}[N] = \bigcup_{N' \in \text{in}(N)} \mathbf{X}[N']. \quad (5.4)$$

The nodes with $\text{out}(N) = \emptyset$ are called the roots or output nodes of the PC. We assume that the roots have full scope, i.e., $\mathbf{X}[N] = \mathbf{X}$. Usually, we assume that the PC has a single root, representing the model (5.1).

We see that the definition of PCs is quite straightforward: We aim to represent a multivariate distribution function $p: \mathcal{X} \mapsto [0, \infty]$ and do this with a neural network whose first layer consists of distribution functions over smaller scopes, and whose higher layers contains nonnegatively weighted sums and products. Since the distribution nodes are nonnegative and the sum weights are nonnegative, the whole function will be nonnegative. Of course, PCs

according to Definition 12 might not be normalized, hence they define a distribution function up to the normalization constant

$$\mathcal{Z} = \int p(\mathbf{x}) d\mathbf{x}, \quad (5.5)$$

which we assume to be finite and non-zero. An example of a PC is shown on Example 7.

5.1.1 Decomposability and Smoothness

The central motivation of PCs is that probabilistic inference remains tractable, i.e., so at least we should be able to compute marginals and conditionals. However, PCs according to Definition 12 are actually intractable. In particular, the normalization constant (5.5) and any other marginals will be pretty hard to compute. We are still missing some “secret sauce” for tractability, namely, *structural constraints*. The two most important structural constraints used in PCs are *decomposability* and *smoothness*.

Definition 13 (Decomposability). *A product node P in a PC is called decomposable if its input nodes have disjoint scopes, i.e.,*

$$\forall N, N' \in \text{in}(P), N \neq N' : \mathbf{X}[N] \cap \mathbf{X}[N'] = \emptyset.$$

A PC is decomposable if all its product nodes are decomposable.

Definition 14 (Smoothness). *A sum node S in a PC is called smooth if all its input nodes have the same scope, i.e.,*

$$\forall N, N' \in \text{in}(S) : \mathbf{X}[N] = \mathbf{X}[N'].$$

Note that by (5.4) a smooth sum node has the same scope as any of its input nodes. A PC is smooth if all its sum nodes are smooth.

Note that the PC in Example 7 is decomposable and smooth. Smoothness is a bit of a cosmetic property, as a non-smooth PC can either be rendered smooth or it turns out that the normalization constant is ∞ , hence the PC is not a valid representation of a distribution. Hence, for now we can just safely assume that any PC is smooth. Decomposability will turn out to be a key property for tractable inference, in particular marginals and conditionals. But first let us better understand what these two structural properties imply.

5.1.2 PCs as Hierarchical Mixtures

An immediate and convenient consequence of decomposability and smoothness is that the normalization constant of a PC is 1 if we (i) use normalized input distributions (i.e. their normalization constant is 1) and (ii) use normalized sum weights, i.e., for each S , additionally to requiring $\theta_{SN} \geq 0$ we also require $\sum_{N \in \text{in}(S)} \theta_{SN} = 1$, so that S computes a *convex combination* of its inputs. More precisely, it will turn out that *each* node in a PC is a normalized distribution over its scope and that the whole PC just forms a hierarchical mixture model.

We show this property by induction over a *topological order* of the PC nodes V . Since the computational graph is acyclic and directed, there exists at least one topological order of the nodes

$$N_1 \prec N_2 \prec \dots \prec N_{|V|}, \quad (5.6)$$

such that it holds for each N_i that all nodes in $\text{in}(N_i)$ have an index smaller than i . In other words, order (5.6) is a topological order if arrows can only point from “left to right”. The existence of at least one topological order is equivalent to the fact that the PC graph $\mathcal{G}$ is directed and acyclic.

We can assume that the input distributions are the first $|\mathbf{D}|$ nodes in this order. Since we assume that they are already properly normalized distributions, their normalization constant is 1. This is the induction basis.

Next assume any node N_i with $i > |\mathbf{D}|$, which is either a sum or product node. We assume, by induction, that all nodes $N_1, \dots, N_{i-1}$ have normalization constant 1 and show that in this case also N_i has normalization constant 1. First note that all $\text{in}(N_i)$ come before N_i in the order and have therefore a normalization constant of 1.

Assume first that $N_i = P$ is a product. For simplicity, we assume that P has only two inputs N and N' . The case for more inputs is immediate. Further, let us rename the scopes of the two input nodes as $\mathbf{X}[N] = \mathbf{Y}$ and $\mathbf{X}[N'] = \mathbf{Z}$ and recall that decomposability means that $\mathbf{Y} \cap \mathbf{Z} = \emptyset$. Then the product computes

$$P(\mathbf{x}) = N(\mathbf{y}) N'(\mathbf{z}). \quad (5.7)$$

Hence, since N and N' are correctly normalized distributions, P is just a *factorized* distribution, assuming independence among $\mathbf{Y}$ and $\mathbf{Z}$, see Section 3.5. In particular, its normalization constant is 1 since

$$\int \int P(\mathbf{x}) d\mathbf{y} d\mathbf{z} = \int \int N(\mathbf{y}) N'(\mathbf{z}) d\mathbf{y} d\mathbf{z} = \overbrace{\left(\int N(\mathbf{y}) d\mathbf{y} \right)}^{=1} \overbrace{\left(\int N'(\mathbf{z}) d\mathbf{z} \right)}^{=1} = 1. \quad (5.8)$$

If, on the other hand, N_i is a sum node, it computes

$$S(\mathbf{x}) = \sum_{N \in \text{in}(S)} \theta_{SN} N(\mathbf{x}). \quad (5.9)$$

Since by induction hypothesis all $\text{in}(S)$ are correctly normalized distributions, and since the weights satisfy $\theta_{SN} \geq 0$ and $\sum_{N \in \text{in}(S)} \theta_{SN} = 1$ and the sum node is smooth, (5.9) is just a *mixture distribution*, i.e. a convex combination of distribution functions. This in turn is always a correctly normalized distribution. In particular, when we compute the normalization constant, we see that

$$\int S(\mathbf{x}) d\mathbf{x} = \int \sum_{N \in \text{in}(S)} \theta_{SN} N(\mathbf{x}) d\mathbf{x} = \sum_{N \in \text{in}(S)} \theta_{SN} \overbrace{\int N(\mathbf{x}) d\mathbf{x}}^{=1} = \sum_{N \in \text{in}(S)} \theta_{SN} = 1. \quad (5.10)$$

Consequently, any node in a decomposable and smooth PC is a correctly normalized distribution over its scope, either by construction (D), or because it is a factorized distribution (P) or a mixture distribution (S), whose factors (respectively mixture components) are given by other PC nodes. We can figure that a PC structure is just a blueprint, crafting larger distributions out of smaller ones, like Lego blocks.

An important thing to note is that, even though products represent factorized distributions and hence independence between the scopes of their input nodes, the overall PC distribution does *not* show these independence properties. This fact stems from the mixture nodes, as a mixture of factorized distributions does (usually) not factorize. One extreme example are Gaussian mixtures⁴ whose Gaussian components have diagonal covariance matrices and hence factorize into univariate Gaussians. Even these models are known to be universal approximators of arbitrary densities, even though each of their components are individually assuming *complete independence* among all random variables.

Decomposability and smoothness together with normalized sum weights yield a very nice interpretation of PCs as a *hierarchical mixture model*. However, an immediate question is whether we loose something when assuming normalized sum weights: Are PCs with unnormalized weights more powerful, i.e., can we represent more distributions with it? The answer to this question is negative [44]: Any decomposable and smooth PC with unnormalized weights can be converted into a PC with identical structure but normalized weights, while representing the exact same distribution. As a warning, this procedure is not simply renormalizing the sum weights, at least not independently. Rather, it performs a

⁴Note that a Gaussian mixture models are just a special case of PC with a single sum node, whose input distributions are all Gaussian.

bottom up sweep over the PC, and normalizes them in turn while compensating the scaling it does to higher nodes. At any rate, however, this result gives us license to always assume that sum weights are normalized, without sacrificing any modeling power.

5.2 Marginalization

The most important consequence of decomposability and smoothness is that PCs allow to compute any marginal distribution in *linear time* of the network size, i.e. the number of edges $|E|$. In order for this to hold true we require that the input distributions $\mathbf{D}$ are themselves tractable, i.e. that they allow to compute any marginal. A decomposable and smooth PC will basically “inherit” this tractability from its input distributions.

One way to ensure this is to use only one-dimensional distributions as input nodes, as marginalizing a single variable will simply deliver the constant 1. Another common choice is to use multivariate Gaussian, parametrized with a D -dimensional mean vector $\boldsymbol{\mu}$ and a $D \times D$ positive definite covariance matrix $\mathbf{C}$:

$$p_{Gauss}(\mathbf{x} | \boldsymbol{\mu}, \mathbf{C}) = \frac{1}{\sqrt{(2\pi)^D |\mathbf{C}|}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \mathbf{C}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right). \quad (5.11)$$

It is a well-known fact that marginals of multivariate Gaussians are again Gaussian. Moreover, these marginals are obtained by simply discarding the entries in $\boldsymbol{\mu}$ and $\mathbf{C}$ corresponding to marginalized random variables.

As we have seen in the last section, sum and product nodes in decomposable and smooth PCs are simply mixtures and factorized distributions of other distributions computed in the PCs. The mechanism for tractable marginalization is that mixtures and factorizations allow to “delegate” marginalization to their mixture components and factors, respectively.

In particular, let S be an arbitrary sum node in a decomposable and smooth PC, computing the mixture distribution

$$S(\mathbf{x}) = \sum_{N \in \text{in}(S)} \theta_{SN} N(\mathbf{x}).$$

Assume we want to marginalize $\mathbf{Z} \subseteq \mathbf{X}$ and let $\mathbf{Y} = \mathbf{X} \setminus \mathbf{Z}$. It holds that

$$\begin{aligned} S(\mathbf{y}) &= \int \sum_{\mathbf{N} \in \text{in}(\mathbf{S})} \theta_{\mathbf{S}\mathbf{N}} N(\mathbf{y}, \mathbf{z}) d\mathbf{z} \\ &= \sum_{\mathbf{N} \in \text{in}(\mathbf{S})} \theta_{\mathbf{S}\mathbf{N}} \int N(\mathbf{y}, \mathbf{z}) d\mathbf{z} \\ &= \sum_{\mathbf{N} \in \text{in}(\mathbf{S})} \theta_{\mathbf{S}\mathbf{N}} N(\mathbf{y}), \end{aligned}$$

meaning that the **marginal of a mixture is just the mixture of marginals**.

On the other hand, let P be an arbitrary product node in a decomposable and smooth PC, computing the factorized distribution

$$P(\mathbf{x}) = N(\mathbf{y}) N'(\mathbf{z}),$$

where we assume that the product node has only two input nodes N, N' (the case for arbitrarily many inputs works the same). Due to decomposability, the scopes of the two nodes, $\mathbf{Y} = \mathbf{X}[N]$ and $\mathbf{Z} = \mathbf{X}[N']$, are disjoint. Assume we want to marginalize a single dimension X_i from P . Due to decomposability, X_i must appear either in $\mathbf{Y}$ or $\mathbf{Z}$, but not both. Say it appears in $\mathbf{Y}$, then the marginal of P is

$$\begin{aligned} \int P(\mathbf{x}) dx_i &= \int N(\mathbf{y}) N'(\mathbf{z}) dx_i \\ &= \left(\int N(\mathbf{y}) dx_i \right) N'(\mathbf{z}), \end{aligned}$$

that is, X_i gets just integrated out only from N since N' is constant with respect to X_i . If we further want to integrate out another random variable X_j which appears in $\mathbf{Z}$ we get

$$\begin{aligned} \int \int P(\mathbf{x}) dx_i dx_j &= \int \int N(\mathbf{y}) N'(\mathbf{z}) dx_i dx_j \\ &= \left(\int N(\mathbf{y}) dx_i \right) \left(\int N'(\mathbf{z}) dx_j \right), \end{aligned}$$

since N is now constant with respect to X_j . Consequently, when we want to integrate out arbitrarily many random variables, the single-dimensional integrals just distribute to the factor where the corresponding variables appear, or in other words, the **marginal of a factorized distribution is just the factorization of the marginals**.

Thus, as long as the mixture components of sum nodes and the factors of product nodes “know” how they can do marginalization, the sum and product nodes just need to do their

“usual job”. When we want to take a marginal of a PC, we apply the integrals to the root. Since the root is either a sum or product node, we can reduce its marginal to the marginals of its input nodes, which are either distributions nodes or again sum and/or product nodes. In the former case we are done, in the latter case we again reduce marginalization to the input nodes of the input nodes. In that way, we recursively delegate the marginalization downwards until the input distributions are reached. For these, however, we were assuming how to do marginalization in the first place. We can summarize this principle by the definition of a marginal PC.

Definition 15 (Marginal PC). *Let a decomposable and smooth PC $\mathcal{G} = (V, E)$ representing a joint distribution $p_{\mathbf{X}}$ be given, and let $\mathbf{Y}, \mathbf{Z}$ be a partition of $\mathbf{X}$, i.e., $\mathbf{Y} \cap \mathbf{Z} = \emptyset$, $\mathbf{Y} \cup \mathbf{Z} = \mathbf{X}$. Define another PC $\mathcal{G}' = (V', E')$ which is an exact copy of $\mathcal{G}$, except that each $D \in V$ is replaced with a marginal distribution $D' \in V'$, with $\mathbf{Z} \cap \mathbf{X}[D]$ being marginalized out. In the case that $\mathbf{Z} \cap \mathbf{X}[D] = \mathbf{X}[D]$, i.e. that all random variables are marginalized from the input distribution, $D' \equiv 1$. The PC $\mathcal{G}' = (V', E')$ is called the $\mathbf{Y}$ -marginal PC of $\mathcal{G}$.*

From the considerations above it follows that the $\mathbf{Y}$ -marginal PC computes the **exact** marginal $p_{\mathbf{Y}}$. Note that the marginal PC might now contain constants 1 as input nodes, so that strictly speaking it is not a PC according to Definition 12. However, we might either interpret these constant input nodes as “distribution over empty sets of random variables” or we can prune them in linear time from the marginal PC, making it a proper PC according to Definition 12.

5.3 Conditioning

With marginalization at hand, we also know how to compute conditionals, which are just a ratio between the joint and a marginal:

$$p(\mathbf{y}|\mathbf{z}) = \frac{p(\mathbf{y}, \mathbf{z})}{p(\mathbf{z})} = \frac{p(\mathbf{y}, \mathbf{z})}{\int p(\mathbf{y}, \mathbf{z}) d\mathbf{y}} \quad (5.12)$$

This requires that we compute two circuit evaluations, one for the joint and one for the marginal. We can do better and construct a *conditional PC* representing the conditional distribution explicitly. The insight here is, like for marginalization, that sum and product nodes allow to delegate the conditioning operations to their input nodes.

Similarly as for marginalization, we require that conditioning is tractable for the input distributions. This is easy for one-dimensional input distributions $D(X)$, as the only possible conditional is to condition on the single variable delivering $\frac{D(x)}{D(x)} = 1$, which is basically a

distribution over an “empty set of random variables,” conditional on $X = x$. For multivariate Gaussians (5.11), conditionals are available closed form. Specifically, let a Gaussian over $\mathbf{Y}, \mathbf{Z}$ with parameters $\boldsymbol{\mu}$ and $\mathbf{C}$ be given. The conditional over $\mathbf{Y}$ given $\mathbf{Z} = \mathbf{z}$ is again Gaussian with mean and covariance

$$\boldsymbol{\mu}_{\mathbf{Y}|\mathbf{z}} = \boldsymbol{\mu}_{\mathbf{Y}} + \mathbf{C}_{\mathbf{YZ}}\mathbf{C}_{\mathbf{ZZ}}^{-1}(\mathbf{z} - \boldsymbol{\mu}_{\mathbf{Z}}) \quad \mathbf{C}_{\mathbf{Y}|\mathbf{z}} = \mathbf{C}_{\mathbf{YY}} - \mathbf{C}_{\mathbf{YZ}}\mathbf{C}_{\mathbf{ZZ}}^{-1}\mathbf{C}_{\mathbf{ZY}}, \quad (5.13)$$

where $\boldsymbol{\mu}_{\mathbf{Y}}$ and $\boldsymbol{\mu}_{\mathbf{Z}}$ are the subvectors of $\boldsymbol{\mu}$ containing the means corresponding to $\mathbf{Y}$ and $\mathbf{Z}$, respectively, $\mathbf{C}_{\mathbf{YY}}$ and $\mathbf{C}_{\mathbf{ZZ}}$ are the squared submatrices of $\mathbf{C}$ containing the (co-)variances corresponding to $\mathbf{Y}$, and $\mathbf{Z}$, respectively, and $\mathbf{C}_{\mathbf{YZ}}$ and $\mathbf{C}_{\mathbf{ZY}}$ are rectangular submatrices of $\mathbf{C}$, containing the covariances between $\mathbf{Y}$ and $\mathbf{Z}$.

Let S be an arbitrary sum node in a decomposable and smooth PCs, computing the distribution

$$S(\mathbf{x}) = \sum_{\mathbf{N} \in \text{in}(S)} \theta_{\mathbf{SN}} \mathbf{N}(\mathbf{x}),$$

where without loss of generality we assume that $\mathbf{X}[S] = \mathbf{X}$. Furthermore, also without loss of generality, we assume that the PC uses normalized sum weights and that each node is consequently a normalized distribution, as discussed in Section 5.1.2

Assume we want to compute the conditional distribution $p_{\mathbf{Y}|\mathbf{Z}}$ where $\mathbf{Y}, \mathbf{Z}$ is some partition of $\mathbf{X}$. We can use the fact that the conditional is proportional to the joint and write:

$$S(\mathbf{y}|\mathbf{z}) \propto \sum_{\mathbf{N} \in \text{in}(S)} \theta_{\mathbf{SN}} \mathbf{N}(\mathbf{y}, \mathbf{z}) \quad (5.14)$$

$$= \sum_{\mathbf{N} \in \text{in}(S)} \overbrace{\theta_{\mathbf{SN}} \mathbf{N}(\mathbf{z})}^{=: \tilde{\theta}_{\mathbf{SN}}} \mathbf{N}(\mathbf{y}|\mathbf{z}) \quad (5.15)$$

$$= \sum_{\mathbf{N} \in \text{in}(S)} \tilde{\theta}_{\mathbf{SN}} \mathbf{N}(\mathbf{y}|\mathbf{z}). \quad (5.16)$$

In (5.16) we see that the conditional of S is proportional to a linear combination of the conditionals of its inputs nodes. The weights $\tilde{\theta}_{\mathbf{SN}} = \theta_{\mathbf{SN}} \mathbf{N}(\mathbf{z})$ are nonnegative but do not sum to one, while the conditionals $\mathbf{N}(\mathbf{y}|\mathbf{z})$ are properly normalized distributions. Hence, the proportionality factor must between (5.16) and $S(\mathbf{y}|\mathbf{z})$ must be $\sum_{\mathbf{N} \in \text{in}(S)} \tilde{\theta}_{\mathbf{SN}}$, i.e.,

$$S(\mathbf{y}|\mathbf{z}) = \sum_{\mathbf{N} \in \text{in}(S)} \frac{\theta_{\mathbf{SN}} \mathbf{N}(\mathbf{z})}{\sum_{\mathbf{N}'} \theta_{\mathbf{SN}'} \mathbf{N}'(\mathbf{z})} \mathbf{N}(\mathbf{y}|\mathbf{z}),$$

which means that the **the conditional of a mixture is a mixture of the conditionals**, with modified mixture weights $\bar{\theta}_{SN'} := \frac{\theta_{SN} N(\mathbf{z})}{\sum_{N'} \theta_{SN'} N'(\mathbf{z})}$.

On the other hand consider a product node

$$P(\mathbf{x}) = N(\mathbf{x}_1) N'(\mathbf{x}_2),$$

where we again assume that the product has only two inputs with scopes $\mathbf{X}_1$ and $\mathbf{X}_2$. The more general case with more than two inputs is straightforward. Let again $\mathbf{Y}, \mathbf{Z}$ be some partition of $\mathbf{X}$ and assume we want to compute $P(\mathbf{y}|\mathbf{z})$. Define $\mathbf{Y}_1 = \mathbf{Y} \cap \mathbf{X}_1$, $\mathbf{Z}_1 = \mathbf{Z} \cap \mathbf{X}_1$, $\mathbf{Y}_2 = \mathbf{Y} \cap \mathbf{X}_2$, $\mathbf{Z}_2 = \mathbf{Z} \cap \mathbf{X}_2$. The conditional is given as

$$P(\mathbf{y}|\mathbf{z}) = \frac{P(\mathbf{y}, \mathbf{z})}{P(\mathbf{z})},$$

which, since the marginal of a product is the product of the marginals, is

$$P(\mathbf{y}|\mathbf{z}) = \frac{N(\mathbf{y}_1, \mathbf{z}_1) N'(\mathbf{y}_2, \mathbf{z}_2)}{N(\mathbf{z}_1) N'(\mathbf{z}_2)} = N(\mathbf{y}_1 | \mathbf{z}_1) N'(\mathbf{y}_2 | \mathbf{z}_2),$$

i.e., **the conditional of a product is the product of the conditionals**.

Hence, similar as for marginalization, we can start at the root of a PC and ask the question what is its conditional distribution. If it is a sum or product node, we have a simple recipe to express its conditional again as a sum and product node of the conditionals of its input distributions. This “delegation” of the conditioning operator happens recursively until we reach the input distributions, for which we assumed tractable conditionals in the first place. We can summarize these insights with the definition of the conditional PC.

Definition 16 (Conditional PC). *Let a decomposable and smooth PC $\mathcal{G} = (V, E)$ representing a joint distribution $p_{\mathbf{X}}$ be given, and let $\mathbf{Y}, \mathbf{Z}$ be a partition of $\mathbf{X}$, i.e., $\mathbf{Y} \cap \mathbf{Z} = \emptyset$, $\mathbf{Y} \cup \mathbf{Z} = \mathbf{X}$. Let a state $\mathbf{z}$ be given. For each $N \in V$, let $N(\mathbf{z})$ be the evaluation of the marginal PC, see Definition 15.*

Define another PC $\mathcal{G}' = (V', E')$ which is an exact copy of $\mathcal{G}$, except that each $D \in V$ is replaced with its corresponding conditional distribution D' over $\mathbf{Y} \cap \mathbf{X}[D]$, conditioned on $\mathbf{z}[D]$. In the case that $\mathbf{Z} \cap \mathbf{X}[D] = \mathbf{X}[D]$, i.e. that all random variables are conditioned on, $D' \equiv 1$.

Furthermore, replace for each sum node S the weights as

$$\theta_{SN} \leftarrow \frac{\theta_{SN} N(\mathbf{z})}{\sum_{N'} \theta_{SN'} N'(\mathbf{z})}$$

The PC $\mathcal{G}' = (V', E')$ is called the $\mathbf{z}$ -conditional PC of $\mathcal{G}$.

Due to the considerations above, the $\mathbf{z}$ -conditional PC represents the exact conditional $p_{\mathbf{Y}|\mathbf{z}}$ of the original PC.

5.4 Expectations and Covariances

We see that decomposable and smooth PCs have excellent properties in terms of probabilistic inference, as the two core routines—marginalization and conditioning—can be computed exactly and efficiently. Furthermore, decomposability and smoothness also allow to compute the expectation (vector) and covariance (matrix) of a PC.

Expectation of PC. Following a similar argument as for marginalization, it can be shown that $\mathbb{E}[\mathbf{X}]$ can be computed exactly and efficiently when $p_{\mathbf{X}}$ is represented as a PC. This follows from the fact that the expectation of a mixture is the mixture of expectations:

$$\mathbb{E}_S[\mathbf{X}] = \int \left(\sum_{N \in \text{in}(S)} \theta_{SN} N(\mathbf{x}) \right) \mathbf{x} d\mathbf{x} = \sum_{N \in \text{in}(S)} \theta_{SN} \int N(\mathbf{x}) \mathbf{x} d\mathbf{x} = \sum_{N \in \text{in}(S)} \theta_{SN} \mathbb{E}_N[\mathbf{X}], \quad (5.17)$$

and hence the expectation of a sum node is recursively given via the expectations of the sum's input nodes.

Furthermore, it is easily shown that the expectation of a factorized distribution is given as the concatenation of the expectations of the factors. Hence, also the expectation of a product node is recursively given via the expectations of the product's input nodes.

This translates into a simple algorithm formulated as a network pass over the PC:

- For each input distribution D , compute $\mathbb{E}_D[\mathbf{X}[D]]$, i.e. the expectation of the distribution node over its scope.
- For each sum node compute its expectation as in (5.17).
- For each product node, compute the expectation as the concatenation of the expectations of its input nodes.

In contrast to marginalization and conditioning, here the messages passed through the network are vector-valued. The output of this procedure, the vector computed at the root node, will be exactly the expectation of the PC.

Covariances. Furthermore, the covariance of a PC can be computed efficiently, again via a recursive argument translating into a network pass. Considering first a sum node computing the mixture

$$S(\mathbf{x}) = \sum_{k=1}^K \theta_k N_k(\mathbf{x}), \quad (5.18)$$

where $|\mathbf{in}(S)| = K$ and we have enumerated the inputs to S as $N_1, \dots, N_K$ using an arbitrary but fixed order. A key technique in mixture models is to interpret them as *latent variable models*.⁵ In particular, (5.18) can be interpreted as the marginal distribution of an augmented model including a latent categorical variable Z taking values in $\{1, \dots, K\}$

$$p_{\mathbf{X},Z}(\mathbf{x},k) = p_Z(k) p_{\mathbf{X}|Z}(\mathbf{x}|k), \quad (5.19)$$

where $p_Z(k) := \theta_k$ and $p_{\mathbf{X}|Z}(\mathbf{x}|k) := N_k(\mathbf{x})$. Indeed, marginalizing Z from (5.19) yields exactly the original mixture (5.18).

The latent variable interpretation allows us to express the covariance $\mathbf{cov}[\mathbf{X}]$ via the *law of total covariance*:

$$\mathbf{cov}[\mathbf{X}] = \mathbb{E}_Z[\mathbf{cov}[\mathbf{X} | Z]] + \mathbf{cov}_Z[\mathbb{E}[\mathbf{X} | Z]]. \quad (5.20)$$

Here $\mathbf{cov}[\mathbf{X} | Z = k]$ is just the covariance of the k^{th} input node to the sum node, which we assume to be tractable. Hence the first term in (5.20) is simply the mixture of these covariances:

$$\mathbb{E}_Z[\mathbf{cov}[\mathbf{X} | Z]] = \sum_k \theta_k \mathbf{cov}[\mathbf{X} | Z = k]. \quad (5.21)$$

Furthermore, $\mathbb{E}[\mathbf{X} | Z = k]$ is just the expectation of the k^{th} input node, which is computed as described in the previous paragraph. By denoting $\boldsymbol{\mu}_k := \mathbb{E}[\mathbf{X} | Z = k]$ and the expectation of the sum node $\boldsymbol{\mu} := \sum_k \theta_k \boldsymbol{\mu}_k$, we get that the second term in (5.20) is given as

$$\mathbf{cov}_Z[\mathbb{E}[\mathbf{X} | Z]] = \sum_k \theta_k (\boldsymbol{\mu}_k - \boldsymbol{\mu})^T (\boldsymbol{\mu}_k - \boldsymbol{\mu}). \quad (5.22)$$

Hence, the covariance matrix of a sum node is recursively given via the covariances of its inputs, as in (5.18).

Furthermore, the covariance matrix of a factorized distribution is simply a block-diagonal matrix, whose on-diagonal blocks are given by the covariances of the factors and the remaining entries are zero (due to independence). Here, without loss of generality, we assume

⁵This theme translates to PCs as well, as I have studied in [42].

that the random variables in the scopes of input nodes are “neighboring” in the covariance matrix—if they were not, the rows and columns would be permuted in some way.

In total, this again leads to an algorithm formulated as a network pass over the PC, where the messages passed through the network are matrix-valued.

- For distribution nodes compute covariance matrices directly from input distributions (e.g., covariance parameters of Gaussian nodes).
- For sum nodes compute the covariance as in (5.20).
- For product nodes construct a block-diagonal covariance matrix from input covariances:

$$\text{cov}[\mathbf{X}] = \begin{pmatrix} \text{cov}[\mathbf{X}_1] & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \text{cov}[\mathbf{X}_2] & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & \text{cov}[\mathbf{X}_n] \end{pmatrix}.$$

5.5 Determinism and Most Probable Explanation

So far, we were assuming only decomposability and smoothness, which already brought us far. Another powerful structural property is *determinism*, which, in combination with decomposability, enables efficient computation of a *most probable explanation* (MPE), i.e. most likely joint assignment of variables:

$$MPE = \arg \max_{\mathbf{x}} p(\mathbf{x}). \quad (5.23)$$

Determinism in PCs is defined as follows.

Definition 17 (Determinism). *A sum node S in a PC is called deterministic if at most one input node is non-zero for any given complete assignment $\mathbf{x}$:*

$$\forall \mathbf{x}: |\{N \in \text{in}(S) : N(\mathbf{x}) > 0\}| \leq 1. \quad (5.24)$$

A PC is deterministic if all its sum nodes are deterministic.

Determinism and decomposability allow to compute an MPE with a single forward pass through the PC. Like for the other inference routines above, this procedure follows by

recursion. First, consider a deterministic sum node S , for which we want to compute

$$\arg \max_{\mathbf{x}} S(\mathbf{x}) = \arg \max_{\mathbf{x}} \sum_{N \in \text{in}(S)} \theta_{SN} N(\mathbf{x}). \quad (5.25)$$

Due to determinism at most one input node is non-zero, for any $\mathbf{x}$, hence we can replace the summation in (5.25) by a maximum

$$\arg \max_{\mathbf{x}} S(\mathbf{x}) = \arg \max_{\mathbf{x}} \max_{N \in \text{in}(S)} \theta_{SN} N(\mathbf{x}), \quad (5.26)$$

which commutes with the maximum over $\mathbf{x}$:

$$\arg \max_{\mathbf{x}} S(\mathbf{x}) = \arg \max_{N \in \text{in}(S)} \max_{\mathbf{x}} \theta_{SN} N(\mathbf{x}) \quad (5.27)$$

$$= \arg \max_{N \in \text{in}(S)} \theta_{SN} \max_{\mathbf{x}} N(\mathbf{x}). \quad (5.28)$$

Thus, if we know how to compute an MPE for each input node N , the MPE of S is given by selecting the MPE of the input node that maximizes $\theta_{SN} N(\mathbf{x})$.

Similarly, consider a decomposable product node $P(\mathbf{x}) = \prod_{N \in \text{in}(P)} N(\mathbf{x})$. It is easy to see that the product is maximized by maximizing individually each input node. Hence the MPE of a decomposable product is simply the concatenation of the MPEs of its input nodes. Consequently, MPE inference can be implemented as a simple forward pass through the PC, where messages are MPE solutions (vectors) together with the maxima they achieve:

- For each distribution node, compute the MPE assignment (which is assumed to be tractable).
- For each sum node, select the MPE of the input node that maximizes the weighted density evaluated at its MPE.
- For each product node, combine the MPE assignments from its inputs.

Determinism has other attractive properties as well. In particular, it allows to compute a fast computation of (global) maximum likelihood parameters [41] and a closed-form solution for Bayesian structure scores [80].

5.6 Further Structural Properties and Tractable Inferences

We see that PCs achieve tractability by exploiting structural properties such as decomposability, smoothness, and determinism. Further structural constraints enable additional efficient

inference routines and operations, extending the applicability of PCs to complex probabilistic queries [71].

A strengthened notion of decomposability is *structured decomposability*, which requires that two product nodes with identical scopes partition their scope identically. Structured decomposability allows to construct tractable circuits representing operations such as products and quotients of circuits, powers of circuits, and logarithms of circuits, which are essential for “advanced” inference queries, in particular combinations of probability and logic.

Two structured-decomposable circuits with identical structure—but different parameters—are called *compatible*, meaning they can be efficiently combined into a new structured-decomposable circuit without introducing exponential complexity. Some circuits are *omni-compatible*, which are compatible with *any* other smooth and decomposable circuit over the same scope $\mathbf{X}$. Omni-compatibility often arises naturally, such as in fully-factorized models or mixtures thereof. (Omni-)compatible PCs support various tractable inference routines beyond marginalization and conditioning, in particular exact computation of complex information-theoretic measures such as Kullback–Leibler divergence, cross-entropy, and mutual information.

See [71] for a systematic overview of these structural properties and the inferences they entail.

5.7 Overview of Contributions

In the first part of this habilitation thesis, I highlight the role of **probability as an excellent core language for AI**: representations of joint distributions (probabilistic models) shall be interpreted as knowledge base, capturing both dependencies between random variables and their uncertainties. This knowledge base is used for probabilistic reasoning, by combining various core inference routines, such as marginalization, conditioning, expectations and most probable explanation.

While probability constitutes a simple, elegant and optimal calculus of reasoning, the downside is that **probabilistic inference is a computational nightmare**. Probabilistic circuits (PCs)—the central topic of this theses—are in a nutshell an elegant approach to probabilistic modeling as they ensure that **inference remains tractable**, that is, computable exactly in polynomial time. The key to tractability lies in structural constraints, such as smoothness, decomposability, determinism, and structural decomposability, where different combinations of constraints lead to different tractable inference routines.

In conclusion of the first part, I provide a brief overview of 17 key papers I have published in the area of PCs in the last ten years, which are then provided in Part II.

5.7.1 Foundations and Theory of PCs

Part of my early work around PCs was focused on foundational aspects. I was first exposed to these ideas via *sum-product networks* (SPNs) [46],⁶ which were proposed as a generalization of *arithmetic circuits* (ACs) [14].⁷ The main proposed innovation in SPNs was that they were learned directly from data, that they did use *unnormalized sum weights* and that they generalized the notion of decomposability to the notion of *consistency*. Thus, a natural question was how much these generalizations extend the modelling capabilities.

In [44, 40] I have shown that they are actually quite vacuous: in particular, it turned out that any smooth and decomposable PC with unnormalized sum weights can be transformed into PC with the same structure and *normalized* sum weights, still representing the same distribution. This shows that using unnormalized weights does not improve the model power. The transformation of sum weights can be done with a single sweep over the PC and hence is also quite efficient. Further, I show in this paper that consistency is not significantly improving the model power either. In particular, any smooth and consistent PC can be transformed into a smooth and decomposable PC by increasing the size of the PC at most by a factor linear in the number of random variables.⁸

In [42], I addressed the interpretation of PCs as *hierarchical latent variable models*, similar as finite mixture models (e.g. Gaussian mixture models) can be interpreted as latent variable models with a single latent categorical variable. This interpretation is important as it establishes a structured view on PCs, enabling the derivation of an exact ancestral sampling algorithm, the classical expectation-maximization algorithm, a sound interpretation of an approximate MPE algorithm proposed in [46], and the interpretation of PCs as a Bayesian network (BN) [28] involving latent variables. The latter can be interpreted as a “decompilation algorithm”, converting a PC into a BN, which we further studied in [8]. The latent variable interpretation in PCs can further be exploited as a representation of data, as we have studied in [75].

Parts of the contributions in [42] also appeared in [40]. A further paper worth mentioning is with Trapp et al. [66], where we proposed a safe semi-supervised learning algorithm for PCs, i.e. using PCs as classifiers trained on both labelled and unlabelled data, with the guarantee that the performance on labelled data can only increase when adding unlabelled data.

⁶Sum-product networks are the special case of smooth and decomposable PC.

⁷Arithmetic circuits are basically equivalent to smooth, decomposable and deterministic PCs.

⁸In order to show that one model class is significantly more expressive than another, one needs to show “exponential separation”, i.e. that there exists a family of distributions with polynomial representation size in the first model class and exponential size in the second.

5.7.2 Bayesian Approaches with PCs

Any parametric machine learning model can be rendered into a Bayesian version: rather than optimizing some loss function with respect to the model parameters one rather (i) equips the parameters with a prior distribution and (ii) infers a parameter posterior given data via Bayes rule. Advantages of Bayesian approaches are that they are generally less prone to overfitting and represent model uncertainty explicitly.

In [74] we proposed a Bayesian inference algorithm for PC parameters based on Gibbs sampling. Moreover, the leaves of PCs were generalized to mixtures over various exponential distributions representing *different data types*, where inference over the data types was also included in the Gibbs sampler. This, together with the latent variable interpretation, allowed to perform automatic recognition of the data type in an “anonymous data table” and further yielded highly-interpretable model, both by its structure and the tractable inference queries PCs facilitate.

In [67] we used a similar Bayesian inference algorithm by Gibbs sampling, but extended the learning to *PC structure*, allowing us to learn both parameters and structure in a single Bayesian setting.

A major drawback of Bayesian approaches in PCs is that they are, to a certain extent, in conflict with tractability: while smooth and decomposable PCs allow tractable marginalization and conditioning, the Bayesian version of them does not. However, including the additional property of determinism and the use of factorized Dirichlet priors on the sum weights allows to perform Bayesian averages in closed form. In [80] this property was used to derive *Bayesian structure scores* [23] for deterministic PCs. These scores are a principled score to compare different model structures in light of observed data and allow to learn an optimal structure in the infinite data limit. While these scores were well-established in the graphical models literature [28, 23], our paper was—and to the time of writing this thesis still is—the only principled structure score proposed for PCs.

5.7.3 Deep Learning and PCs

Deep learning has without doubt been the most impactful technology in AI up to now. While the field of deep learning is truly massive, its basic principle is rather simple: (i) express the learning task as a function approximation problem, (ii) construct an expressive function approximator via a (large) computational graph, i.e. a composite function whose structure is described by directed acyclic graph over (differentiable) predefined functions, (iii) use automatic differentiation to compute gradients of model parameters, which is used in (iv) gradient-based optimization of a suitable loss function governed by the training data. Many

tricks of the trade contributed to the success of deep learning we are witnessing today, but a particular game changer was the development of dedicated libraries such as Tensorflow [1] and PyTorch [38], which allow to implement and train deep models with ease.

PCs are also computational graphs and can be understood as special case of neural networks, whose input layer contain non-linear functions (densities) and whose higher layers either contain product or linear units. Additionally, as discussed before, we apply some suitable subset of structural constraints in order to unlock tractable inference queries of our choice.

It is hence natural to use PCs as deep learning models as well. In particular, they might be implemented in deep learning frameworks, benefit from GPU-acceleration and automatic differentiation, and might be easily interfaced with other deep learning models. However, my first attempts to implement PCs on Tensorflow were naive, as I implemented each individual node as a separate node. This led to huge and sparsely connected computational graph of small operations, which is a very GPU-unfriendly design.

The first decent implementation of PCs on Tensorflow were RAT-SPNs [45], which vectorized input distributions and sum nodes, and product nodes were replaced with “unrolled” outer products. The resulting computational graph had much fewer nodes and a higher degree of parallelism. This implementation allowed to scale PCs to relatively large dataset such as (fashion)MNIST, but it was still about 50 times slower than a comparable multi-layer perceptron (MLP). The main reason for this bottleneck was the “scattered” and sparse graph induced by the PCs’ structural constraints.

Taking lesson from these insights I developed *einsum networks* (EiNets) a further accelerated implementation for PCs [43]. The main idea in this paper was to further increase the degree of parallelism by “summarizing” several vectorized sum nodes into a bigger layer (tensor) and implementing all sum-product operations from one layer to the next with one monolithic call to the *einsum*-function.⁹ EiNets were now approximately en par with MLPs of comparable size,¹⁰ and scaled to datasets sizes which were previously out of reach for PCs. This central principle in this paper indicates a strong connection between PCs and tensor factorization, which has further been addressed in [31]. Another interesting contributions in this paper was a simple implementation of EM which relied entirely on standard backpropagation.

⁹Einsum is implementing the Einstein sum-product contraction. It allows to represent all linear algebra operations as a parametrized call to *einsum*. It is readily implemented and optimized in deep learning frameworks.

¹⁰My implementation, just using standard *einsum*, was just about 5 times slower than MLPs. With further optimization, e.g. implementing dedicated GPU-kernels, this difference can be made even.

As mentioned above, implementing PCs on deep learning frameworks makes them operationally compatible to neural networks. In particular, one can now construct *conditional PCs* [56, 57], representing *conditional distributions* $p(\mathbf{y}|\mathbf{x})$, by devising a PC structure over $\mathbf{Y}$, whose parameters $\boldsymbol{\theta}(\mathbf{x})$ depend functionally on some context $\mathbf{x}$. One is completely free how to represent $\boldsymbol{\theta}(\mathbf{x})$ and might use arbitrary neural networks for this purpose. As long as the PC satisfies the structural constraints with respect to $\mathbf{Y}$, it allows, for any $\mathbf{x}$, tractable inference in the distribution $p(\mathbf{y}|\mathbf{x})$. Several of such conditional PC can also be combined into block-wise autoregressive models, for example as expressive generative models over images.

Furthermore, PCs lead to other interesting applications in deep learning. In particular, if one allows for unconstrained (negative and positive) sum-weights and replaces the input densities with arbitrary non-linear functions, one yields a (non-probabilistic) circuit or *decomposable neural network* [63]. This architecture is less constrained than PCs and has interesting applications. In particular, in [63], we exploited the fact that expectations over diagonal (independent) Gaussians inputs can be tractably computed in deconets which can be used in exact robustness guarantees concerning adversarial attacks [10].

5.7.4 Hybrid Models and Inference

PCs make a fantastic promise, as probabilistic inference, which is generally NP-hard in most representations of joint distributions, is easy when the distribution is represented as a PC. The catch is that this property weakens the expressive power of PCs. One seems to be confronted with a dichotomy: Do we want tractable inference and hence go with PCs (or some other tractable model)? Or do we need expressive power and hence go with highly expressive models such as VAEs, GANs, Flows, auto-regressive models, etc., and accept the fact that inference needs to be approximated?

It is suggestive that this is a *false dichotomy* and that there actually exists an interesting (and largely unexplored) middle ground of *hybrid models*: tractable where possible, approximate where required. Some of my work has explored some options to achieve such hybrid models.

In [62] we have proposed a variation of the *attend-infer-repeat* (AIR) system [18] using *PCs as a sub-module of an intractable model*. Specifically, the AIR system is a method for Bayesian visual scene analysis, putting a prior on the scene, in particular on the number of objects, their category, scale, position, etc. Further, it uses a rendering procedure to specify a likelihood of an observed image given a latent scene. The goal is to infer the latent scene given an observed image, naturally formulated as Bayesian posterior inference. The AIR system solves this task by using a recurrent neural network to infer objects sequentially

and removing them from the scene. In the generative model, the different object categories are represented with object-specific VAEs [26]. Hence, the entire model contains several thousand latent random variables to be inferred, in particular random variables describing the scene descriptions, pixels and VAE-codes of objects. The inference method of AIR was entirely based on amortized variational inference.

In [62] we replaced the object models with PCs and devised a technique to marginalize them from the posterior, leaving just a few variables for the scene description to be inferred. Hence, the entire system was not “fully tractable”, but we solved large parts of the inference problem via tractable inference and inferred the rest with variational inference. Here, the intuition is that intractable models over few variables are “more tractable” than intractable models over many variables.¹¹ The main result in our paper was faster convergence of scene inference—even though we use the slow implementation of [45]—as well as more stable results of the variational inference routine.

Rather than taking PCs as sub-modules of intractable models one might also use *intractable modes as sub-modules of PCs*. Specifically, in [64] we used VAEs [26] as PC leaves. This is syntactically valid, as VAEs are merely infinite mixture distributions. We further showed that stochastic ELBO estimates, which are used as learning signal in vanilla VAEs, can be used as replacement of log-likelihoods for the PC leaves, yielding an ELBO estimate for the hybrid PC/VAE model and hence a sound learning objective. The main insight is, again, that the VAE leaves can be much smaller than a regular VAE (in particular their latent code can be of smaller dimensionality), since they model only part of the random variables. Hence, the resulting model is a “collection of many small intractable models, glued together via a tractable model”, which intuitively might yield a simpler inference problem.¹² In our experiments we indeed showed that our hybrid model learned faster and achieved higher log-likelihoods than vanilla VAEs.

Basically the same idea was pursued in [68] but using Gaussian processes (GPs) [79] as leaves rather than VAEs. GPs actually do admit tractable inference, but require cubic time and quadratic memory in the number of samples, which becomes a notorious bottleneck for a large number of samples. Besides using sparse inducing point techniques [47], the most popular technique to address this problem is to use a divide-and-conquer approach, by splitting the data points in subsets, performing GP inference on these subsets and aggregating the results. In [68], we also followed this principle by devising a PC structure over local GP experts. Unlike other divide-and-conquer approaches, our model was a sound and well-

¹¹Note that this is a rule-of-thumb line of reasoning and making this theoretically concrete requires much deeper thought.

¹²Again, this is an qualitative argument.

defined probabilistic model and still admitted *exact* inference, resulting in more faithful inferences than the baselines.

In [13] and [19] we explored a further interesting principle to combine tractable models with intractable ones. The basic idea in these papers was to augment the PC formalism with a new type of node: an integral node. Formally, an integral node is equipped with a continuous latent random variable (or random vector) distributed according to some simple distribution, like Gaussian. PC parameters below this integral node then depend functionally on this latent variable, yielding an infinite mixture of PCs. Note that VAEs can also be described in this framework and just contain a single integral node. Unlike VAEs, in [13, 19] we leveraged *numerical integration* techniques (quadratures), for which approximation guarantees can be derived. To this end, the dimensionality of the latent continuous variables need to be kept small enough, as numerical integration scales badly to large dimension. The main outcome of these papers was that PCs with integral nodes outperform classical PCs.

5.7.5 PCs and Symbolic Models

PCs have a close connection to (tractable) logical circuits [16, 14], which enables neuro-symbolic approaches by combining the two [2]. Moreover, they have natural connections to other machine learning models and symbolic methods.

In [12], we established a connection between probabilistic circuits and decision trees. In particular, decision trees can be transformed into PCs by transforming each decision node into a sum node over products of the sub-trees below the decision node and indicator nodes representing the respective decisions. Additionally,

- one attaches weights to the generated sum nodes given by the normalized counts of samples split by the decision, conditional on all decisions above the decision node;
- one also augments the classifier associated with each of the leaves—a conditional distribution over the prediction target—by multiplying it with a (simple) joint distribution over features using the samples associated to the leaf.

The resulting PC represents a joint distribution $p(\mathbf{x}, y)$, over both features $\mathbf{x}$ and target y rather than a conditional model $p(y|\mathbf{x})$. The conditional distribution derived from this joint, however, is under certain conditions exactly the same as the original decision tree classifier. Hence, learning a decision tree and transforming it into a PC can be seen as augmenting it into a full joint distribution, still yielding the same classifier in a “backward compatible way”. Advantages of this procedure are, among others, the possibility to treat missing input features by PC marginalization and to detect outliers by monitoring the probability of input

features $p(\mathbf{x})$. Furthermore, also random forests can be augmented to generative models by re-interpreting them as mixtures of PCs obtained from the individual decision trees.

In [30] we established a connection between *knowledge graph embeddings* (KGEs) and PCs. Knowledge graphs are a prominent symbolic representation of domain knowledge. Formally, they are directed multi-graphs whose nodes are *entities* and whose edges correspond to *predicates* between entities. The entity where an edge starts is called *subject* and the entity where it ends is called *object*. The entire knowledge graph can be understood as a collection of subject-predicate-object (SPO) triplets. Typical tasks in knowledge graphs are *link prediction* (which is important because most knowledge graphs are severely incomplete) and inferring new predicates, e.g. by learning and applying rules among predicates

Knowledge graphs are typically used in machine learning applications by leveraging KGEs, obtained by constructing vector-valued representations for entities and predicates, which can be combined into an SPO embedding by applying some function to the S-, P- and O-embeddings. A wide range of functions have been proposed in literature, where many of them are using some sum-product form, making them reminiscent to PCs. In [30], we made this connection explicit and devised ways to modify common KGEs into bona-fide PCs. The advantages of this approach are, among others, that standard maximum likelihood training can be used to learn KGEs (instead of perhaps less principled techniques such as pseudo-likelihood and contrastive techniques), that our approach allows to sample from the distribution over SPO triplets, faster inference for triplets with missing entities or predicates and the possibility to encode hard logical constraints in KGEs.

Finally, in [78] we used logical circuits and probabilistic inference to construct the first exact approach for *soft analytic side-channel attacks* (SASCA) [76]. SASCA highlights vulnerability of cryptography algorithms (such as AES) to physical information leaks, where any kind of physical side-channel might be used, such as power traces, heat dissipation, and hardware timing. SASCA assumes that an attacker has access to a physical copy of a device running a cryptographic algorithm. By querying the device with many random inputs of plain message and key, the attacker can learn so-called *templates*, i.e. probabilistic models mapping from side-channel to intermediate computations in the algorithm. These “soft guesses” can be integrated by exploiting the logical structure of the cryptographic algorithm, which can be formulated as probabilistic inference in a factor graph [32], inferring the secret key from the observed side channel.

The standard approach in SASCA is to use *loopy belief propagation* for inference, which has few theoretical guarantees and whose inference quality is hard to assess. In [78], we used a knowledge compilation approach [16] to compile large parts of the factor graph into a circuit representation and leveraged PC inference for *exact inference* about the key. The result

was a dramatic increase of success rate and a higher tolerance against noise, which highlights further vulnerabilities against protected implementations of cryptographic algorithms.

References

- [1] Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., and Zheng, X. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.
- [2] Ahmed, K., Teso, S., Chang, K.-W., Van den Broeck, G., and Vergari, A. (2022). Semantic probabilistic layers for neuro-symbolic learning. *Advances in Neural Information Processing Systems*, 35:29944–29959.
- [3] Arjovsky, M., Chintala, S., and Bottou, L. (2017). Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. PMLR.
- [4] Berger, J. O. (1990). Statistical decision theory. In *Time Series and Statistics*, pages 277–284. Springer.
- [5] Billingsley, P. (1995). *Probability and Measure*. Wiley-Interscience.
- [6] Bingham, E., Chen, J. P., Jankowiak, M., Obermeyer, F., Pradhan, N., Karaletsos, T., Singh, R., Szerlip, P., Horsfall, P., and Goodman, N. D. (2019). Pyro: Deep universal probabilistic programming. *Journal of machine learning research*, 20(28):1–6.
- [7] Bishop, C. M. and Nasrabadi, N. M. (2006). *Pattern recognition and machine learning*, volume 4. Springer.
- [8] Butz, C., Oliveira, J. S., and Peharz, R. (2020). Sum-product network decompilation. In *International Conference on Probabilistic Graphical Models*, pages 53–64. PMLR.
- [9] Choi, Y., Vergari, A., and Van den Broeck, G. (2020). Probabilistic circuits: A unifying framework for tractable probabilistic models. *UCLA*. URL: <http://starai.cs.ucla.edu/papers/ProbCirc20.pdf>, page 6.
- [10] Cohen, J., Rosenfeld, E., and Kolter, Z. (2019). Certified adversarial robustness via randomized smoothing. In *international conference on machine learning*, pages 1310–1320. PMLR.
- [11] Cooper, G. F. (1990). The computational complexity of probabilistic inference using bayesian belief networks. *Artificial intelligence*, 42(2-3):393–405.

- [12] Correia, A., Peharz, R., and de Campos, C. P. (2020). Joints in random forests. *Advances in neural information processing systems*, 33:11404–11415.
- [13] Correia, A. H., Gala, G., Quaeghebeur, E., de Campos, C., and Peharz, R. (2023). Continuous mixtures of tractable probabilistic models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 7244–7252.
- [14] Darwiche, A. (2003). A differential approach to inference in bayesian networks. *Journal of the ACM (JACM)*, 50(3):280–305.
- [15] Darwiche, A. (2009). *Modeling and reasoning with Bayesian networks*. Cambridge university press.
- [16] Darwiche, A. and Marquis, P. (2002). A knowledge compilation map. *Journal of Artificial Intelligence Research*, 17:229–264.
- [17] Dechter, R. and Mateescu, R. (2007). And/or search spaces for graphical models. *Artificial intelligence*, 171(2-3):73–106.
- [18] Eslami, S., Heess, N., Weber, T., Tassa, Y., Szepesvari, D., Hinton, G. E., et al. (2016). Attend, infer, repeat: Fast scene understanding with generative models. *Advances in neural information processing systems*, 29.
- [19] Gala, G., de Campos, C., Peharz, R., Vergari, A., and Quaeghebeur, E. (2024). Probabilistic integral circuits. In *International Conference on Artificial Intelligence and Statistics*, pages 2143–2151. PMLR.
- [20] Ghahramani, Z. (2013). Bayesian non-parametrics and the probabilistic approach to modelling. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 371(1984):20110553.
- [21] Ghahramani, Z. (2015). Probabilistic machine learning and artificial intelligence. *Nature*, 521(7553):452–459.
- [22] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative adversarial nets. *Advances in neural information processing systems*, 27.
- [23] Heckerman, D., Geiger, D., and Chickering, D. M. (1995). Learning bayesian networks: The combination of knowledge and statistical data. *Machine learning*, 20:197–243.
- [24] Jaynes, E. T. (1957). Information theory and statistical mechanics. *Physical review*, 106(4):620.
- [25] Jaynes, E. T. (2003). *Probability theory: The logic of science*. Cambridge university press.
- [26] Kingma, D. P., Welling, M., et al. (2019). An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392.
- [27] Kisa, D., Van den Broeck, G., Choi, A., and Darwiche, A. (2014). Probabilistic sentential decision diagrams. In *Fourteenth International Conference on the Principles of Knowledge Representation and Reasoning*.

- [28] Koller, D. and Friedman, N. (2009). *Probabilistic graphical models: principles and techniques*. MIT press.
- [29] Larochelle, H. and Murray, I. (2011). The neural autoregressive distribution estimator. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 29–37. JMLR Workshop and Conference Proceedings.
- [30] Loconte, L., Di Mauro, N., Peharz, R., and Vergari, A. (2023). How to turn your knowledge graph embeddings into generative models. *Advances in Neural Information Processing Systems*, 36:77713–77744.
- [31] Loconte, L., Mari, A., Gala, G., Peharz, R., de Campos, C., Quaeghebeur, E., Vessio, G., and Vergari, A. (2024). What is the relationship between tensor factorizations and circuits (and how can we exploit it)? *TMLR, accepted*.
- [32] Loeliger, H.-A. (2004). An introduction to factor graphs. *IEEE Signal Processing Magazine*, 21(1):28–41.
- [33] MacKay, D. J. (1995). Bayesian neural networks and density networks. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 354(1):73–80.
- [34] MacKay, D. J. (2003). *Information theory, inference and learning algorithms*. Cambridge university press.
- [35] Martens, J. and Medabalimi, V. (2014). On the expressive efficiency of sum product networks. *arXiv preprint arXiv:1411.7717*.
- [36] Olteanu, D. and Schleich, M. (2016). Factorized databases. *ACM SIGMOD Record*, 45(2):5–16.
- [37] Papamakarios, G., Nalisnick, E., Rezende, D. J., Mohamed, S., and Lakshminarayanan, B. (2021). Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64.
- [38] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc.
- [39] Pearl, J. (1988). *Probabilistic reasoning in intelligent systems: networks of plausible inference*. Morgan kaufmann.
- [40] Peharz, R. (2015). *Foundations of sum-product networks for probabilistic modeling*. PhD thesis, PhD thesis, Graz University of Technology.
- [41] Peharz, R., Gens, R., and Domingos, P. (2014). Learning selective sum-product networks. In *31st International Conference on Machine Learning (ICML2014)*.
- [42] Peharz, R., Gens, R., Pernkopf, F., and Domingos, P. (2016). On the latent variable interpretation in sum-product networks. *IEEE transactions on pattern analysis and machine intelligence*, 39(10):2030–2044.

- [43] Peharz, R., Lang, S., Vergari, A., Stelzner, K., Molina, A., Trapp, M., Van den Broeck, G., Kersting, K., and Ghahramani, Z. (2020a). Einsum networks: Fast and scalable learning of tractable probabilistic circuits. In *International Conference on Machine Learning*, pages 7563–7574. PMLR.
- [44] Peharz, R., Tschitschek, S., Pernkopf, F., and Domingos, P. (2015). On theoretical properties of sum-product networks. In *Artificial Intelligence and Statistics*, pages 744–752. PMLR.
- [45] Peharz, R., Vergari, A., Stelzner, K., Molina, A., Shao, X., Trapp, M., Kersting, K., and Ghahramani, Z. (2020b). Random sum-product networks: A simple and effective approach to probabilistic deep learning. In *Uncertainty in Artificial Intelligence*, pages 334–344. PMLR.
- [46] Poon, H. and Domingos, P. (2011). Sum-product networks: A new deep architecture. In *2011 IEEE International Conference on Computer Vision Workshops (ICCV Workshops)*, pages 689–690. IEEE.
- [47] Quinonero-Candela, J. and Rasmussen, C. E. (2005). A unifying view of sparse approximate gaussian process regression. *Journal of machine learning research*, 6(Dec):1939–1959.
- [48] Rahman, T., Kothalkar, P., and Gogate, V. (2014). Cutset networks: A simple, tractable, and scalable approach for improving the accuracy of chow-liu trees. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2014, Nancy, France, September 15-19, 2014. Proceedings, Part II 14*, pages 630–645. Springer.
- [49] Rezende, D. and Mohamed, S. (2015). Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR.
- [50] Rezende, D. J., Mohamed, S., and Wierstra, D. (2014). Stochastic backpropagation and approximate inference in deep generative models. In *International conference on machine learning*, pages 1278–1286. PMLR.
- [51] Rosenthal, J. S. (2006). *A First look at rigorous probability theory*. World Scientific Publishing Company.
- [52] Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- [53] Russell, S. and Norvig, P. (2010). *Artificial Intelligence: A Modern Approach*. Prentice Hall, 3 edition.
- [54] Shafer, G. and Vovk, V. (2008). A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9(3).
- [55] Shannon, C. E. (1948). A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423.
- [56] Shao, X., Molina, A., Vergari, A., Stelzner, K., Peharz, R., Liebig, T., and Kersting, K. (2020). Conditional sum-product networks: Imposing structure on deep probabilistic architectures. In *International Conference on Probabilistic Graphical Models*, pages 401–412. PMLR.

- [57] Shao, X., Molina, A., Vergari, A., Stelzner, K., Peharz, R., Liebig, T., and Kersting, K. (2022). Conditional sum-product networks: Modular probabilistic circuits via gate functions. *International Journal of Approximate Reasoning*, 140:298–313.
- [58] Simon, H. A. (1983). *Why Should Machines Learn?*, pages 25–37. Springer Berlin Heidelberg.
- [59] Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25.
- [60] Song, Y. and Kingma, D. P. (2021). How to train your energy-based models. *arXiv preprint arXiv:2101.03288*.
- [61] Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. (2020). Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*.
- [62] Stelzner, K., Peharz, R., and Kersting, K. (2019). Faster attend-infer-repeat with tractable probabilistic models. In *International Conference on Machine Learning*, pages 5966–5975. PMLR.
- [63] Subramani, P. S., Kamath, G., Peharz, R., et al. (2021). Exact and efficient adversarial robustness with decomposable neural networks. In *The 4th Workshop on Tractable Probabilistic Modeling*.
- [64] Tan, P. L. and Peharz, R. (2019). Hierarchical decompositional mixtures of variational autoencoders. In *International Conference on Machine Learning*, pages 6115–6124. PMLR.
- [65] Tomczak, J. M. (2021). *Deep Generative Modeling*. Springer.
- [66] Trapp, M., Madl, T., Peharz, R., Pernkopf, F., and Trappl, R. (2017). Safe semi-supervised learning of sum-product networks. *arXiv preprint arXiv:1710.03444*.
- [67] Trapp, M., Peharz, R., Ge, H., Pernkopf, F., and Ghahramani, Z. (2019). Bayesian learning of sum-product networks. *Advances in neural information processing systems*, 32.
- [68] Trapp, M., Peharz, R., Pernkopf, F., and Rasmussen, C. E. (2020). Deep structured mixtures of gaussian processes. In *International conference on artificial intelligence and statistics*, pages 2251–2261. PMLR.
- [69] Van de Meent, J.-W., Paige, B., Yang, H., and Wood, F. (2018). An introduction to probabilistic programming. *arXiv preprint arXiv:1809.10756*.
- [70] Van den Oord, A., Kalchbrenner, N., Espeholt, L., Vinyals, O., Graves, A., et al. (2016). Conditional image generation with pixelcnn decoders. *Advances in neural information processing systems*, 29.
- [71] Vergari, A., Choi, Y., Liu, A., Teso, S., and Van den Broeck, G. (2021). A compositional atlas of tractable circuit operations for probabilistic inference. *Advances in Neural Information Processing Systems*, 34:13189–13201.

- [72] Vergari, A., Choi, Y., and Peharz, R. (2022). Probabilistic circuits: Representations, inference, learning and applications. In *Tutorial at the The Thirty-Sixth Annual Conference on Neural Information Processing Systems (NeurIPS)*.
- [73] Vergari, A., Choi, Y., Peharz, R., and Van den Broeck, G. (2020). Probabilistic circuits: Representations, inference, learning and applications. In *Tutorial at the The 34th AAAI Conference on Artificial Intelligence*.
- [74] Vergari, A., Molina, A., Peharz, R., Ghahramani, Z., Kersting, K., and Valera, I. (2019). Automatic bayesian density analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 5207–5215.
- [75] Vergari, A., Peharz, R., Di Mauro, N., Molina, A., Kersting, K., and Esposito, F. (2018). Sum-product autoencoding: Encoding and decoding representations using sum-product networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.
- [76] Veyrat-Charvillon, N., Gérard, B., and Standaert, F.-X. (2014). Soft analytical side-channel attacks. In *Advances in Cryptology–ASIACRYPT 2014: 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, ROC, December 7–11, 2014. Proceedings, Part I 20*, pages 282–296. Springer.
- [77] Vincent, P. (2011). A connection between score matching and denoising autoencoders. *Neural computation*, 23(7):1661–1674.
- [78] Wedenig, T., Nagpal, R., Cassiers, G., Mangard, S., and Peharz, R. (2024). Exact soft analytical side-channel attacks using tractable circuits. In *Proceedings of the 41st International Conference on Machine Learning*, pages 52472–52483.
- [79] Williams, C. K. and Rasmussen, C. E. (2006). *Gaussian processes for machine learning*, volume 2. MIT press Cambridge, MA.
- [80] Yang, Y., Gala, G., and Peharz, R. (2023). Bayesian structure scores for probabilistic circuits. In *International Conference on Artificial Intelligence and Statistics*, pages 563–575. PMLR.